```
# ===== backend/app/main.py =====
"""探海灵眸 Oceanus — 后端应用入口。
启动流程（lifespan）：
    1. 初始化日志
    2. 连接 Redis
    3. 启动 MQTT 订阅
    4. ACK 判重水位恢复（MQTT 客户端可用后从 t_task_ack 重建，
       失败/超时降级继续启动，结果写入 app.state 供 /health 观测）
    5. 启动派单消费者（Redis Streams）
     6. 启动定时任务（补派、报表聚合）
    关闭时逆序释放资源。
 """
from __future__ import annotations
import asyncio
import sys
import time
from contextlib import asynccontextmanager
from typing import Any, AsyncGenerator
# Windows 上 aiomqtt（基于 paho-mqtt）依赖事件循环的 add_reader/add_writer，
#   而 Windows 默认的 ProactorEventLoop 不支持这两个 API（抛 NotImplementedError），
#   导致 MQTT 客户端永远连不上、表现为「Operation timed out」。
#   必须在任何事件循环创建之前（uvicorn 启动前）切到 Selector 事件循环。
if sys.platform == "win32":
    asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())
from fastapi import FastAPI
from fastapi.responses import JSONResponse
from fastapi.middleware.cors import CORSMiddleware
from loguru import logger
from app.api.v1 import api_router
from app.core.config import settings, validate_production_settings
from app.core.deps import close_redis, get_redis
from app.db.session import dispose_engine
from app.middleware.response import register_exception_handlers, trace_id_middleware
#: ACK 判重水位启动恢复的超时上界（秒）。
#: 恢复来自 t_task_ack 的规范回执与设备 ACK 水位；数据库不可用 / 表缺失 /
#: 查询异常一律降级为 warning 继续启动，绝不让启动无限期挂起或失败。
ACK_TRACKER_REBUILD_TIMEOUT_SECONDS = 5.0
async def recover_ack_tracker(
    tracker: Any,
    *,
    session_factory: Any = None,
    timeout: float = ACK_TRACKER_REBUILD_TIMEOUT_SECONDS,
) -> dict[str, Any]:
    """启动阶段从 t_task_ack 恢复 ACK 判重水位。
    tracker 为平台侧 app.mqtt.ack.AckTracker（MQTT 客户端实例
    持有）。实现即调用
    TaskAckRepository.rebuild_ack_tracker(tracker)（能力方法）。
    恢复边界（如实声明，不伪装完整恢复）：
    - duplicate / out_of_order 判定重启后可恢复（规范回执 +
      设备 ACK 水位来自账本）；
```

```
- late 判定依赖发布时登记的 expires_at（t_task_ack 不存）——
    「发布后未 ACK」的命令重启后仍不可判 late（内存登记表固有边界）。
四项要求：
1. 降级不失败：数据库不可用 / 表不存在 / 查询异常 → warning 并
    继续启动，绝不抛异常；
2. 幂等：重复启动不叠加、不报错（rebuild_ack_tracker 对已存在
    的规范回执只判重不覆盖，水位取 max）；
3. 耗时上界：整个重建包在 asyncio.wait_for 内，超时即放弃并降级；
4. 可观测：返回 {status, rows, elapsed_ms, detail}，由调用方
    写入 app.state.ack_tracker_recovery 供 /health 展示，同时打日志。
session_factory 可注入（测试用）；缺省用全局 get_session_factory。
"""
result: dict[str, Any] = {
    "status": "ok",
    "rows": 0,
    "elapsed_ms": 0.0,
    "detail": "",
}
if tracker is None:
    result.update(
        status="degraded",
        detail="ack_tracker 不可用（MQTT 客户端未启动），跳过判重水位恢复",
    )
    logger.warning(f"[启动] ACK 判重水位恢复降级：{result['detail']}")
    return result
if session_factory is None:
    from app.db.session import get_session_factory as _default_factory
    session_factory = _default_factory
started = time.monotonic()
async def _rebuild() -> int:
    async with session_factory()() as session:
        from app.repositories import TaskAckRepository
        return await TaskAckRepository(session).rebuild_ack_tracker(tracker)
try:
    rows = await asyncio.wait_for(_rebuild(), timeout=timeout)
    result["rows"] = int(rows)
    result["detail"] = f"从 t_task_ack 重建 {rows} 条规范回执"
    logger.info(f"[启动] ACK 判重水位恢复：{result['rows']} 条规范回执已重建")
except asyncio.TimeoutError:
    result.update(
        status="degraded",
        detail=f"重建超过上界 {timeout}s（已取消），降级继续启动",
    )
    logger.warning(f"[启动] ACK 判重水位恢复超时（>{timeout}s），降级继续启动")
except Exception as exc:   # noqa: BLE001
    result.update(
        status="degraded",
        detail=f"{type(exc).__name__}: {exc}",
    )
     logger.warning(f"[启动] ACK 判重水位恢复失败（{exc}），降级继续启动")
```

```
    finally:
        result["elapsed_ms"] = round((time.monotonic() - started) * 1000.0, 3)
    return result
@asynccontextmanager
async def lifespan(app: FastAPI) -> AsyncGenerator[None, None]:
    """应用生命周期管理。"""
    validate_production_settings(settings)
    logger.info(f"[启动] {settings.app_name} 环境={settings.app_env}")
    # 1. Redis
    try:
        redis = await get_redis()
        await redis.ping()
        logger.info(f"[启动] Redis 已连接 {settings.redis_host}:{settings.redis_port}")
    except Exception as exc:   # noqa: BLE001
        await close_redis()
        logger.warning(f"[启动] Redis 连接失败 ({exc}) , 队列功能不可用")
    # 2. MQTT 订阅
    # import 必须放在 try 里: 把 import 留在 try 外面, 就等于宣称
    #   「aiomqtt 缺失时能降级」但实际会直接崩在启动阶段——
    #   降级路径必须把「依赖不存在」和「依赖存在但连不上」一视同仁。
    mqtt_client = None
    try:
        from app.mqtt.client import mqtt_client
        from app.mqtt.handlers import HANDLERS
        await mqtt_client.start(HANDLERS)
    except Exception as exc:   # noqa: BLE001
        logger.warning(f"[启动] MQTT 启动失败 ({exc}) , 设备上报通道不可用")
    # 2.5 ACK 判重水位恢复
    # MQTT 客户端可用后, 从 t_task_ack 重建 AckTracker 判重水位 (duplicate /
    # out_of_order 判定重启后可恢复) 。数据库不可用 / 表缺失 / 查询异常 →
    # 降级 warning 继续启动; 恢复有超时上界; 结果写入 app.state 供 /health
    # 观测, 不得静默。
    ack_recovery_result = None
    if mqtt_client is not None:
        try:
            ack_recovery_result = await recover_ack_tracker(mqtt_client.ack_tracker)
        except Exception as exc:   # noqa: BLE001
            ack_recovery_result = {
                "status": "degraded",
                "rows": 0,
                "elapsed_ms": 0.0,
                "detail": f"{type(exc).__name__}: {exc}",
            }
            logger.warning(f"[启动] ACK 判重水位恢复异常 ({exc}) , 降级继续启动")
    app.state.ack_tracker_recovery = ack_recovery_result
    # 3. 后台任务 (派单消费者 + 补派定时器 + 报表聚合)
    background_tasks: list[asyncio.Task[None]] = []
    if settings.background_workers_enabled:
        try:
            from app.services.consumer import dispatch_consumer, pending_dispatcher
```

```
            from app.services.report import daily_report_worker


            background_tasks.append(asyncio.create_task(dispatch_consumer()))
            background_tasks.append(asyncio.create_task(pending_dispatcher()))
            background_tasks.append(asyncio.create_task(daily_report_worker()))
            logger.info("[启动] 后台任务已启动（派单消费者 + 补派定时器 + 报表聚合）")
        except Exception as exc:  # noqa: BLE001
            logger.warning(f"[启动] 后台任务启动失败（{exc}）")
    else:
        logger.info("[启动] 后台任务已按配置关闭（BACKGROUND_WORKERS_ENABLED=false）")
    logger.info(f"[启动] 完成，接口文档：http://localhost:8000/docs")
    yield
    # 关闭
    # 每步都独立 try：启动阶段降级掉的组件（mqtt_client 可能是 None），
    # 关闭时不能反过来把进程打崩。
    logger.info("[关闭] 正在释放资源...")
    for task in background_tasks:
        task.cancel()
    if background_tasks:
        await asyncio.gather(*background_tasks, return_exceptions=True)
    try:
        from app.services.sim import simulation_manager
        await simulation_manager.stop_all()
    except Exception as exc:  # noqa: BLE001
        logger.warning(f"[关闭] 仿真引擎释放异常（{exc}）")
    if mqtt_client is not None:
        try:
            await mqtt_client.stop()
        except Exception as exc:  # noqa: BLE001
            logger.warning(f"[关闭] MQTT 释放异常（{exc}）")
    for name, closer in (("Redis", close_redis), ("数据库引擎", dispose_engine)):
        try:
            await closer()
        except Exception as exc:  # noqa: BLE001
            logger.warning(f"[关闭] {name} 释放异常（{exc}）")
    logger.info("[关闭] 完成")
def create_app() -> FastAPI:
    """创建 FastAPI 应用。"""
    docs_enabled = settings.app_env != "production"
    app = FastAPI(
        title="探海灵眸 Oceanus API",
        description=(
            "海漂垃圾「感知—决策—执行」全链路智能治理系统 · 后端接口\n\n"
            "平台是中枢，不是显示屏：识别到垃圾不是弹框，"
            "而是产生事件 → 触发派单 → 形成可追溯工单。"
        ),
        version="1.0.0",
        docs_url="/docs" if docs_enabled else None,
        redoc_url="/redoc" if docs_enabled else None,
        openapi_url="/openapi.json" if docs_enabled else None,
        lifespan=lifespan,
```

```
)


# 中间件
app.middleware("http")(trace_id_middleware)
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
# 异常处理
register_exception_handlers(app)
# 路由
app.include_router(api_router, prefix=settings.api_v1_prefix)
async def probe_dependencies() -> dict[str, str]:
    """探测运行依赖，供 liveness 与 readiness 共用。"""
    deps: dict[str, str] = {}
    # Redis
    try:
        redis = await get_redis()
        await asyncio.wait_for(redis.ping(), timeout=2.0)
        deps["redis"] = "ok"
    except Exception as exc:   # noqa: BLE001
        await close_redis()
        deps["redis"] = f"down: {type(exc).__name__}"
    # MQTT (只看客户端状态，不做网络探测)
    try:
        from app.mqtt.client import mqtt_client
        deps["mqtt"] = "ok" if mqtt_client.is_connected else "down"
    except Exception as exc:   # noqa: BLE001
        deps["mqtt"] = f"down: {type(exc).__name__}"
    # 数据库
    try:
        from sqlalchemy import text
        from app.db.session import get_session_factory
        async with get_session_factory()() as session:
            # asyncpg 的连接超时由 get_engine() 的 connect_args 控制。
            # 这里不要再用 wait_for 从外部取消：那会触发 asyncpg 的
            # Connection._cancel 清理协程，留下未 await 的 RuntimeWarning。
            await session.execute(text("SELECT 1"))
        deps["database"] = "ok"
    except Exception as exc:   # noqa: BLE001
        deps["database"] = f"down: {type(exc).__name__}"
    return deps
# 健康检查
@app.get("/health", tags=["系统"], summary="存活检查")
async def health() -> dict:
    """存活检查：必须反映真实降级状态。
    /health 保持 HTTP 200，便于观测部分依赖故障时的系统状态；
    编排系统请使用 /ready 决定是否接流量。
```

```
        """

        from app.ws.manager import ws_manager
        deps = await probe_dependencies()
        bad = [k for k, v in deps.items() if v != "ok"]
        status = "ok" if not bad else ("degraded" if len(bad) < len(deps) else "down")
        # ACK 判重水位恢复结果：启动时从 t_task_ack 重建的观测值。
        # 顶层字段、不进入 dependencies：不影响三档 status 语义（既有
        # 契约测试锁定 dependencies 只含 redis/mqtt/database）。
        ack_recovery = getattr(app.state, "ack_tracker_recovery", None)
        return {
            "status": status,
            "app": settings.app_name,
            "env": settings.app_env,
            "ws_connections": ws_manager.connection_count,
            "dependencies": deps,
            "ack_recovery": ack_recovery,
        }
    @app.get("/ready", tags=["系统"], summary="就绪检查")
    async def ready() -> JSONResponse:
        """就绪检查：任一依赖不可用时返回 HTTP 503。"""
        deps = await probe_dependencies()
        bad = [k for k, v in deps.items() if v != "ok"]
        status = "ok" if not bad else ("degraded" if len(bad) < len(deps) else "down")
        payload = {
            "status": status,
            "app": settings.app_name,
            "env": settings.app_env,
            "dependencies": deps,
        }
        return JSONResponse(payload, status_code=200 if not bad else 503)
    @app.get("/", tags=["系统"], summary="服务信息")
    async def root() -> dict:
        return {
            "name": settings.app_name,
            "description": "海漂垃圾智能治理系统后端",
            "docs": "/docs" if docs_enabled else None,
            "api_prefix": settings.api_v1_prefix,
        }
    return app
app = create_app()
if __name__ == "__main__":
    import uvicorn
    uvicorn.run("app.main:app", host="0.0.0.0", port=8000, reload=True)
# ===== backend/app/services/dispatch.py =====
"""派单引擎：平台最核心的业务逻辑。
对应「感知—决策—执行」闭环中的「决策」环节。
五步筛选逻辑：
    1. 可用性过滤（在线 / 电量 / 仓容）
    2. 距离排序（KNN）
    3. 类别匹配加权（顺路复用）
```

```
    4. 选中下发（MQTT + 等 ACK）



    5. ACK 超时处理（换车重派）
防抖：
    同网格区域窗口期内的多个事件合并为一个任务，
    避免机器人被反复派往同一片水域。
"""
from __future__ import annotations
import re
import struct
import uuid
from datetime import datetime, timedelta, timezone
from typing import Any
from loguru import logger
from sqlalchemy.ext.asyncio import AsyncSession
from app.core.config import settings
from app.core.exceptions import NoRobotAvailableError
from app.core.geo import nearest_township
from app.models.device import Device
from app.models.event import Event, EventStatus
from app.models.task import Task, TaskPriority, TaskStatus
from app.repositories import DeviceRepository, EventRepository, TaskRepository
def _gen_task_id() -> str:
    """生成任务编号：tsk_YYYYMMDD_xxxx。"""
    return f"tsk_{datetime.now():%Y%m%d}_{uuid.uuid4().hex[:6]}"
def _utcnow() -> datetime:
    """任务生命周期统一使用 UTC aware 时间。"""
    return datetime.now(timezone.utc)
def _as_utc(value: datetime) -> datetime:
    """兼容历史库里可能出现的 naive 时间戳。"""
    return value if value.tzinfo is not None else value.replace(tzinfo=timezone.utc)
# ---------------------------------------------------------------------
# 坐标解析
# ---------------------------------------------------------------------
_EWKT_POINT_RE = re.compile(
    r"POINT\s*\(\s*(-?\d+(?:\.\d+)?)\s+(-?\d+(?:\.\d+)?)\s*\)", re.IGNORECASE
 )
# EWKB 标志位：0x20000000 = 带 SRID；低 8 位是几何类型（1 = Point）
_EWKB_SRID_FLAG = 0x20000000
_WKB_POINT = 1
def _parse_point(value: Any) -> tuple[float, float] | None:
    """把几何值解析成 `(lng, lat)`；解析不出来返回 None。
    为什么不再发一条 `SELECT ST_X(...)`：
        任务对象手里本来就持有坐标，为取坐标再跑一次数据库是纯浪费；
        更要紧的是——历史写法 `select(ST_X(task.target_location))` 没有
        `where(...)`，一旦 `task.target_location` 是列引用，`.first()`
        拿到的就是全表第一行的坐标，机器人会被派去别人的目标点，
        而全过程不报错。坐标是内存里已经有的值，就在内存里解析。
    支持两种形态：
        - EWKT 字符串 `SRID=4326;POINT(lng lat)`（新创建任务时的内存值）
        - EWKB 字节 / `WKBElement`（从库里读回的值）
```

```
    """



    if value is None:
        return None
    if isinstance(value, str):
        return _parse_ewkt(value)
    raw = getattr(value, "data", None)      # geoalchemy2 的 WKBElement
    if raw is None and isinstance(value, (bytes, bytearray, memoryview)):
        raw = value
    if raw is not None:
        return _parse_ewkb(bytes(raw))
    return None
def _parse_ewkt(text: str) -> tuple[float, float] | None:
    match = _EWKT_POINT_RE.search(text)
    if match is None:
        return None
    return float(match.group(1)), float(match.group(2))
def _parse_ewkb(data: bytes) -> tuple[float, float] | None:
    """解析 (E)WKB 的 Point 类型。只支持点——任务目标点本来就是点。"""
    if len(data) < 5:
        return None
    endian = "<" if data[0] == 1 else ">"
    (gtype,) = struct.unpack_from(endian + "I", data, 1)
    if gtype & 0xFF != _WKB_POINT:
        return None
    offset = 5
    if gtype & _EWKB_SRID_FLAG:
        offset += 4
    if len(data) < offset + 16:
        return None
    x, y = struct.unpack_from(endian + "dd", data, offset)
    return x, y
# ---------------------------------------------------------------------
# 派单收尾（唯一出口）
# ---------------------------------------------------------------------
async def finalize_dispatch(task: Task) -> None:
    """派单成功的统一收尾：下发 MQTT 指令 + 推送看板。
    四个派单入口都必须调用它：
        1. `mqtt.handlers._try_dispatch`：HTTP 上报后的同步派单
        2. `services.consumer._process_entry`：Streams 消费者
        3. `services.consumer.pending_dispatcher`—— 定时补派
        4. `DispatchEngine.assign_pending_tasks`：手动补派 API
    为什么必须收成一处（这是本函数的全部理由）：
        曾经只有前两个入口做了收尾，后两个只创建任务就返回。后果是
        「库里有任务、日志写着补派成功、API 回了 task_ids」，
        而机器人一条指令都没收到，看板也不刷新，任务永远停在
        assigned：不报错、不抛异常、不打日志，是最难查的一类。
        收尾动作集中到一处后，新加入口不可能再各写各的。
    守卫见 `backend/tests/test_dispatch_finalize.py`：
    用 AST 穷举所有 `dispatch_for_event` 调用点，断言每个都接了本函数。
    """
```

```
401    from app.mqtt.client import mqtt_client
402
403
404
405    from app.ws.manager import ws_manager
406    if task.robot_id:
407        point = _parse_point(task.target_location)
408        if point is None:
409            # 拿不到坐标必须留痕：静默跳过会让「机器人为什么没动」无从查起
410            logger.error(
411                f"[派单] 任务 {task.task_id} 的坐标无法解析"
412                f" (类型 {type(task.target_location).__name__}) , "
413                f"仅推送看板、不下发指令"
414            )
415        else:
416            lng, lat = point
417            await mqtt_client.publish_task(
418                robot_id=task.robot_id,
419                task_id=task.task_id,
420                lng=lng,
421                lat=lat,
422                priority=task.priority,
423            )
424    else:
425        logger.warning(f"[派单] 任务 {task.task_id} 没有绑定机器人，跳过下发指令")
426    # 四个字段一个都不能少（契约真源：docs/api.md §「服务端推送消息」）
427    await ws_manager.push_task_update(
428        {
429            "task_id": task.task_id,
430            "event_id": task.event_id,
431            "robot_id": task.robot_id,
432            "status": task.status,
433        }
434    )
435class DispatchEngine:
436    """派单引擎。
437    用法：
438        engine = DispatchEngine(session)
439        result = await engine.dispatch_for_event(event)
440    """
441    def __init__(self, session: AsyncSession) -> None:
442        self.session = session
443        self.devices = DeviceRepository(session)
444        self.events = EventRepository(session)
445        self.tasks = TaskRepository(session)
446    # ------------------------------------------------------------------
447    # 主入口
448    # ------------------------------------------------------------------
449    async def dispatch_for_event(self, event: Event) -> Task | None:
450        """为一条事件尝试派单。
        返回创建的 Task；若合并到已有任务则返回该任务；
        若该事件已有在途任务则返回 None。
        无可用机器人时抛 `NoRobotAvailableError`（不是返回 None）。
```

```
451         这里曾写成「返回 None」，与实现不符——照着 docstring 写调用方
452         就不会 try/except，异常会一路穿到 HTTP 层变成 500。
453         """
454         # 前置：防重复派单
455         if await self.tasks.has_active_task_for_event(event.event_id):
456             logger.info(f"[派单] 事件 {event.event_id} 已有在途任务，跳过")
457             return None
458         # 防抖：同区域窗口内合并
459         merged = await self._try_merge(event)
460         if merged is not None:
461             logger.info(f"[派单] 事件 {event.event_id} 合并至任务 {merged.task_id}")
462             await self._mark_event(event, EventStatus.DISPATCHED)
463             return merged
464         # 五步筛选
465         robot = await self._select_robot(event)
466         if robot is None:
467             logger.warning(f"[派单] 事件 {event.event_id} 无可用机器人，保持待派单")
468             raise NoRobotAvailableError("当前无满足条件的可用机器人（在线/电量/仓容）")
469         task = await self._create_and_assign(event, robot)
470         await self._mark_event(event, EventStatus.DISPATCHED)
471         logger.info(
472             f"[派单] 事件 {event.event_id} → 任务 {task.task_id} → 机器人 {robot.device_id}"
473         )
474         return task
475     # ------------------------------------------------------------------
476     # 内部步骤
477     # ------------------------------------------------------------------
478     async def _try_merge(self, event: Event) -> Task | None:
479         """防抖合并：查找窗口期内附近的活跃任务。
480         合并判断需要知道「当时有没有在途任务」，所以必须放在引擎层而非事件层。
481         """
482         # 取事件坐标
483         lng, lat = await self._event_lnglat(event)
484         if lng is None or lat is None:
485             return None
486         return await self.tasks.find_mergeable_task(
487             lng=lng,
488             lat=lat,
489             window_minutes=settings.dispatch_merge_window_minutes,
490             radius_m=settings.dispatch_merge_radius_meters,
491         )
492     async def _select_robot(self, event: Event) -> Device | None:
493         """五步筛选的前三步：可用性过滤 → 距离排序 → 类别匹配加权。
494         锚点是事件坐标：`find_nearest_available_robots` 用 event_id
495         反查 `t_event.location`。
496         """
497         return await self._select_robot_at_point(
498             event_id=event.event_id,
499             event=event,
500         )
```

```
async def _select_robot_at_point(
    self,
    *,
    event_id: str | None = None,
    lng: float | None = None,
    lat: float | None = None,
    event: Event | None = None,
    exclude_robot_ids: set[str] | None = None,
) -> Device | None:
    """选机器人的唯一实现：从候选池挑一台。
    两种入参二选一：
        - `event_id` + `event`：事件派单，锚点是事件坐标；
        - `lng` / `lat`：人工建单的工单（`event_id IS NULL`），
          锚点直接来自 `t_task.target_location`。
    为什么要把「候选要怎么筛、怎么加权、怎么排序」收在一处：
        这段规则（电量阈值、仓容阈值、类别加权、距离兜底）只有一份时，
        改策略才不会出现「事件派单按新规则、人工补派按旧规则」的静默分叉。
        两条入参路径的差别仅在「锚点从哪来」这一步。
    """
    if event_id is not None:
        candidates = await self.devices.find_nearest_available_robots(
            event_id,
            max_distance_m=settings.dispatch_range_meters,
            limit=5,
            exclude_robot_ids=exclude_robot_ids,
        )
    else:
        if lng is None or lat is None:
            logger.error("[派单] 选机器人时既没有 event_id 也没有坐标，跳过")
            return None
        candidates = await self.devices.find_nearest_available_robots_at(
            lng,
            lat,
            max_distance_m=settings.dispatch_range_meters,
            limit=5,
            exclude_robot_ids=exclude_robot_ids,
        )
    if not candidates:
        return None
    # 过滤出真正可用的（电量 / 仓容）
    usable: list[tuple[Device, float, int]] = []
    for device, distance in candidates:
        meta = device.meta or {}
        battery = int(meta.get("battery", 100))
        bins = meta.get("bins", {}) or {}
        total_usage = sum(float(v) for v in bins.values())
        if battery < settings.dispatch_min_battery:
            logger.debug(f"[派单] {device.device_id} 电量 {battery}% 不足，跳过")
            continue
        if total_usage >= settings.dispatch_max_bin_usage:
```

```
                logger.debug(f"[派单] {device.device_id} 仓容 {total_usage:.0%} 已满，跳过")
                continue
            # 类别匹配加权：正在处理同类别任务的机器人优先（顺路复用）。
            # 人工建单的工单没有 event（也就没有 main_class），无从配对，
            # 一律给普通权重 1：排序退化为「按距离最近」，是合理缺省。
            weight = await self._class_match_weight(event, device.device_id) if event else 1
            usable.append((device, distance, weight))
        if not usable:
            return None
        # 排序：类别匹配优先（0 优于 1），其次距离最近
        usable.sort(key=lambda x: (x[2], x[1]))
        return usable[0][0]
    async def _class_match_weight(self, event: Event, robot_id: str) -> int:
        """类别匹配加权：0=该机器人已在处理同类别任务（更优），1=普通。
        必须两两比对 event 与机器人现有任务的类别。
        这里曾写成 `return 0 if event.main_class == FOAM else 1`——
        收了 `task` 形参却从未使用，于是加权退化成「按事件类别给常量」，
        与机器人手上在跑什么完全无关。后果：
          a. 文档承诺的「同类别顺路复用」从未生效（空驶优化是空的）；
          b. 因为返回的常量对同一 event 的所有候选机器人完全相同，
             排序实际只剩「距离」一维：看起来在工作，实则白算。
        任务的类别取自它关联事件的 `main_class`。
        """
        active_tasks = await self.tasks.list_active_tasks_for_robot(robot_id)
        if not active_tasks:
            return 1   # 手上没任务 → 无复用可言，普通权重
        active_event_ids = [t.event_id for t in active_tasks if t.event_id]
        if not active_event_ids:
            return 1
        classes = await self.events.list_main_classes_for_events(active_event_ids)
        # 该机器人任一在途任务的类别与本次事件相同 → 顺路，加权
        return 0 if event.main_class in classes else 1
    async def _create_and_assign(self, event: Event, robot: Device) -> Task:
        """第四步：创建任务记录并下发（MQTT 下发由 mqtt 层订阅后执行）。"""
        lng, lat = await self._event_lnglat(event)
        now = _utcnow()
        # 优先级取自单一真源 TaskPriority，不在此写 `1 if ... else 3`
        priority = TaskPriority.for_waste_class(event.main_class)
        task = await self.tasks.create(
            task_id=_gen_task_id(),
            event_id=event.event_id,
            robot_id=robot.device_id,
            target_location=f"SRID=4326;POINT({lng} {lat})",
            status=TaskStatus.ASSIGNED,
            priority=priority,
            assigned_at=now,
            township=event.township or (
                nearest_township(lng, lat) if lng is not None and lat is not None else None
            ),
        )
```

```
        # 更新机器人状态（忙碌中）


        meta: dict[str, Any] = dict(robot.meta or {})
        meta["current_task_id"] = task.task_id
        robot.meta = meta
        return task
    async def _mark_event(self, event: Event, status: str) -> None:
        event.status = status
    async def _event_lnglat(self, event: Event) -> tuple[float | None, float | None]:
        """读取事件坐标（PostGIS geometry → 经纬度）。"""
        point = _parse_point(event.location)
        if point is None:
            return None, None
        return point
    # ------------------------------------------------------------------
    # 状态机流转
    # ------------------------------------------------------------------
    async def transition(
        self,
        task: Task,
        target_status: str,
        *,
        robot_id: str | None = None,
        collected_weight: float | None = None,
        review_result: str | None = None,
        remark: str | None = None,
    ) -> Task:
        """任务状态迁移（唯一合法入口）。
        禁止在别处直接改 task.status：所有变更必须经过此方法校验。
        """
        from app.core.exceptions import InvalidStateTransitionError
        current = task.status
        if not TaskStatus.can_transition(current, target_status):
            raise InvalidStateTransitionError(current, target_status)
        now = _utcnow()
        task.status = target_status
        if robot_id is not None:
            task.robot_id = robot_id
        # 时间戳链
        if target_status == TaskStatus.ASSIGNED and task.assigned_at is None:
            task.assigned_at = now
        elif target_status == TaskStatus.NAVIGATING:
            task.ack_at = task.ack_at or now
        elif target_status == TaskStatus.COLLECTING:
            task.started_at = task.started_at or now
        elif target_status == TaskStatus.DONE:
            task.finished_at = now
        if collected_weight is not None:
            task.collected_weight = collected_weight
        if review_result is not None:
            task.review_result = review_result
```

```
        if remark is not None:
            task.remark = remark
        logger.info(f"[状态机] 任务 {task.task_id}: {current} → {target_status}")
        # 任务完成 → 同步事件状态
        if target_status == TaskStatus.DONE and task.event_id:
            event = await self.events.get_by_event_id(task.event_id)
            if event is not None:
                event.status = EventStatus.RESOLVED
        return task
    async def handle_ack_timeout(self, task: Task) -> bool:
        """第五步：ACK 超时处理。
        机器人 15 秒未确认，任务退回待派单池，换下一台。
        返回 True 表示已回退。
        """
        if task.status != TaskStatus.ASSIGNED:
            return False
        if task.assigned_at is None:
            return False
        elapsed = _utcnow() - _as_utc(task.assigned_at)
        if elapsed < timedelta(seconds=settings.dispatch_ack_timeout_seconds):
            return False
        logger.warning(f"[派单] 任务 {task.task_id} ACK 超时 ({elapsed.total_seconds():.0f}s) ，回…
        await self.transition(task, TaskStatus.PENDING, remark="ACK 超时，重新派单")
        return True
    async def assign_pending_tasks(self, limit: int = 10) -> list[Task]:
        """扫描待派单对象并尝试补派（补派 API 与定时任务调用）。
        扫描两类对象，两类都要覆盖：
            1. `t_event` 中状态仍为 `new` 的高优先级事件——
               当初无可用机器人，事件保持 new；机器人上线后由
               `dispatch_for_event` 建一条新任务派出去；
            2. `t_task` 中 `status='pending' AND robot_id IS NULL` 的工单——
               人工建单（`POST /tasks` 不指定机器人）产生的工单。
               它们 `event_id` 为 NULL，没有任何事件可扫，第一类永远扫不到。
        第二类是 2026-09-27 线上验收发现的缺陷（报告 P1-3）：
            人工建单 → 点「触发补派」→ 接口回 `{"dispatched": 0}` 且提示
            「完成补派 0 个任务」，工单永远停在待派单，既不报错也不留日志。
            根因就是这里只扫了事件表。
        两类共用同一处收尾（见下方注释），调用方无需区分来源。
        """
        created: list[Task] = []
        events = await self.events.recent_for_dispatch(limit=limit)
        for event in events:
            if await self.tasks.has_active_task_for_event(event.event_id):
                continue
            try:
                task = await self.dispatch_for_event(event)
                if task is not None:
                    created.append(task)
            except NoRobotAvailableError:
                continue   # 暂无可用机器人，等下一轮
```

```
        # 第二类：人工建单产生的「无事件待派单工单」。


        #   放在第一类之后，让事件派单优先占用机器人：事件由摄像头自动
        #   发现、有明确的时空证据；人工建单是平台判读后的补充指令。
        created.extend(await self._assign_unassigned_pending(limit=limit))
        # 收尾（下发 + 推送）必须在循环之后统一做：
        #   放在循环内的话，一旦后面的事件抛异常触发 rollback，
        #   前面已经下发过的任务会与数据库状态不一致。
        for task in created:
            await finalize_dispatch(task)
        return created
    async def _assign_unassigned_pending(self, limit: int = 10) -> list[Task]:
        """把 `status='pending' AND robot_id IS NULL` 的工单派出去。
        为什么不复用 `dispatch_for_event`：
            那个入口以 event 为中心：防抖合并（`_try_merge`）、
            事件状态回写（`_mark_event`）都挂在它上面。人工建单的工单
            `event_id` 就是空的，走那条路要么被跳过、要么得先伪造一条事件。
            本方法直接改已有工单的 robot_id / status，不新建任务。
        与 `_create_and_assign` 保持等价的赋值口径（robot_id、
        `assigned_at` 重置、机器人 `meta.current_task_id`），
        确保工单无论是自动派还是补派出去的，字段状态都一致。
        """
        tasks = await self.tasks.list_unassigned_pending(limit=limit)
        if not tasks:
            return []
        assigned: list[Task] = []
        taken: set[str] = set()   # 本轮已派出去的机器人，见下
        for task in tasks:
            point = _parse_point(task.target_location)
            if point is None:
                # 坐标取不到必须留痕。`target_location` 是 NOT NULL 列，
                #   走到这里说明几何值解析失败（异常写入 / 新几何类型），
                #   静默 continue 会让人以为「补派功能又坏了」。
                logger.error(
                    f"[补派] 工单 {task.task_id} 的目标坐标无法解析"
                    f" (类型 {type(task.target_location).__name__}) ，跳过"
                )
                continue
            lng, lat = point
            robot = await self._select_robot_at_point(
                lng=lng, lat=lat, exclude_robot_ids=taken
            )
            if robot is None:
                logger.info(
                    f"[补派] 工单 {task.task_id} 附近无可用机器人"
                    f" ({lng:.5f},{lat:.5f}) ，保持待派单"
                )
                continue
            await self.transition(task, TaskStatus.ASSIGNED, robot_id=robot.device_id)
            # 与 `reassign_task` 同理：`transition` 只在 `assigned_at`
            #   为空时才赋值，这里显式写成本轮时间，ACK 超时判定才有起点。
```

```
            task.assigned_at = _utcnow()



            meta: dict[str, Any] = dict(robot.meta or {})
            meta["current_task_id"] = task.task_id
            robot.meta = meta
            taken.add(robot.device_id)
            assigned.append(task)
            logger.info(f"[补派] 工单 {task.task_id} → 机器人 {robot.device_id}")
        return assigned
    async def reassign_task(self, task: Task) -> bool:
        """把 PENDING 任务重新派给一台可用机器人（「换车重派」）。
        与 `handle_ack_timeout` 配套：那一步只把卡住的任务退回 PENDING，
        退回之后必须有人把它再派出去，否则任务只是换个状态继续卡着。
        这两步过去都没有任何调用点，于是文档里「五步筛选」的第五步
        （ACK 超时处理）从未真正生效过——机器人掉线后任务会永久停在
        assigned，且没有任何日志。
        """
        if task.status != TaskStatus.PENDING:
            return False
        if not task.event_id:
            return False
        event = await self.events.get_by_event_id(task.event_id)
        if event is None:
            return False
        robot = await self._select_robot(event)
        if robot is None:
            return False
        await self.transition(task, TaskStatus.ASSIGNED, robot_id=robot.device_id)
        # 必须重置计时起点：`transition` 只在 `assigned_at` 为空时才赋值，
        #   沿用旧值会让这个任务下一轮立刻又被判成超时，
        #   于是每 30 秒空转重派一次，永远收敛不了。
        task.assigned_at = _utcnow()
        await finalize_dispatch(task)
        return True
# ===== backend/app/services/agents/runtime.py =====
"""确定性 Agent Runtime 内核（首版，纯内存，零外部依赖）。
设计目标（手册第 4 节）：
    不依赖数据库 / Redis / MQTT / 外网 / 大模型即可运行测试。
核心入口（手册 3.6）：
    AgentRuntime.run(request)    -> AgentRunResult
    AgentRuntime.cancel(run_id, actor, reason) -> AgentRunResult
    AgentRuntime.resume(run_id)  -> AgentRunResult
    AgentRuntime.status()        -> RuntimeStatus
可注入依赖（手册 3.6）：tool_registry / policy_guard / memory_store /
run_repository / approval_repository / clock / id_factory / runtime_config。
状态机（手册 3.1）：
    created -> planning -> waiting_policy -> waiting_approval
             -> executing -> observing -> verifying
             -> succeeded | failed | cancelled | expired（终态不可再迁移）
确定性规则模式：
    runtime_config.model_available=False（默认）时无模型照常工作；
```

```
    计划由 rule_plan 模板生成，期望由 Expectation 规则判定，全部可复现。



冻结语义：规划步骤的轨迹表示（总控裁定，后续必须沿用）：
    - 一次规划阶段（plan phase）恰好产出一条 plan 步骤，decision_summary
      为摘要式：「计划意图 + 计划动作数 + 关键要点」（手册 3.7 解密预算：
      只保存决策摘要，动作级细节不进轨迹）。
    - 重规划阶段产出一条 replan 步骤（重规划原因摘要）+ 随附一条新的
      plan 步骤（同样是摘要式）。
    - 任何 plan/replan 步骤都只追加、不覆盖历史步骤（手册 3.2）。
解密预算（手册 3.7）：
    轨迹只写 decision_summary（决策摘要）与 input_hash / output_hash，
    记忆只写摘要与业务理由，绝不保存模型私有思维链。
"""
from __future__ import annotations
import logging
import time
import uuid
from dataclasses import dataclass
from datetime import datetime, timedelta, timezone
from typing import Any, Protocol
from app.services.agents.errors import (
    ERROR_CODES,
    AgentError,
    AgentNotFoundError,
    ErrorCode,
    InternalAgentError,
)
from app.services.agents.memory import InMemoryMemoryStore, MemoryEntry, MemoryStore
from app.services.agents.lessons import LessonStore, summarize_hits
from app.services.agents.model import (
    TERMINAL_STATUSES,
    AgentRun,
    AgentRunRequest,
    AgentRunResult,
    AgentStatus,
    AgentStep,
    AgentStepType,
    LEGAL_TRANSITIONS,
    RuntimeConfig,
    RuntimeStatus,
    RuleStep,
)
from app.services.agents.model_adapter import ModelAdapterPlanner, summarize_tools
from app.services.agents.planner import RulePlanner
from app.services.agents.policy import PolicyGuard, RulePolicyGuard
from app.services.agents.repositories import (
    ApprovalRecord,
    ApprovalRepository,
    InMemoryApprovalRepository,
    InMemoryRunRepository,
    RunRepository,
```

```
)



from app.services.agents.schema import sha256_hex
from app.services.agents.tools import ToolDefinition, ToolExecutor, ToolRegistry, ToolResult
logger = logging.getLogger("oceanus.agents")
# ---------------------------------------------------------------------
# 时钟与 ID 工厂（可注入，测试可确定复现）
# ---------------------------------------------------------------------
class Clock(Protocol):
    """可注入时钟：now() 返回业务时间，now_ms() 返回单调毫秒。"""
    def now(self) -> datetime: ...
    def now_ms(self) -> int: ...
class SystemClock:
    """真实时钟（默认）。now() 返回 aware UTC，与 PG TIMESTAMPTZ 语义一致。"""
    def now(self) -> datetime:
        return datetime.now(timezone.utc)
    def now_ms(self) -> int:
        return int(time.time() * 1000)
class FakeClock:
    """假时钟：测试注入，advance() 推进，now()/now_ms() 同源一致。
    now() 返回 aware UTC（与 SystemClock 对齐）：naive 时间戳写入 PG
    TIMESTAMPTZ 会被按会话时区解释，读回后偏移导致审批/过期误判超时。
    """
    def __init__(self, start: datetime | None = None, start_ms: int = 0) -> None:
        start = start or datetime(2026, 1, 1, 0, 0, 0)
        if start.tzinfo is None:
            start = start.replace(tzinfo=timezone.utc)
        self._start = start
        self._ms = start_ms
    def now(self) -> datetime:
        return self._start + timedelta(milliseconds=self._ms)
    def now_ms(self) -> int:
        return self._ms
    def advance(self, ms: int) -> None:
        self._ms += ms
def _as_aware(dt: datetime | None) -> datetime | None:
    """统一为带时区时间用于比较：naive 视为 UTC；None 原样返回。
    时钟（FakeClock/SystemClock）返回 aware UTC；数据库仓储（PG TIMESTAMPTZ）
    读回的记录同样 aware。_as_aware 是对第三方/遗留 naive 值的防御：直接相减
    naive 与 aware 会抛 TypeError（SQLite 不暴露此差异），naive 按 UTC 解释与
    PG 存储语义一致。
    """
    if dt is None:
        return None
    if dt.tzinfo is None:
        return dt.replace(tzinfo=timezone.utc)
    return dt
class IdFactory(Protocol):
    """可注入 ID 工厂：new(prefix) 生成带冻结前缀的 ID。
    协议方法体做参数校验后抛 NotImplementedError：既保证形参被真实
    使用（静态守卫），又保持纯协议语义。
```

```
    """



    def new(self, prefix: str) -> str:
        if not isinstance(prefix, str) or not prefix:
            raise TypeError(f"prefix 必须是非空 str：{prefix!r}")
        raise NotImplementedError("IdFactory.new 未实现")
class SequenceIdFactory:
    """序列 ID 工厂：确定性可复现（测试用，run_0001、stp_0001……）。"""
    def __init__(self, start: int = 1, width: int = 4) -> None:
        self._n = start
        self._width = width
    def new(self, prefix: str) -> str:
        value = self._n
        self._n += 1
        return f"{prefix}{value:0{self._width}d}"
class UuidIdFactory:
    """UUID 截断 ID 工厂（默认，保证跨进程唯一）。"""
    def new(self, prefix: str) -> str:
        return f"{prefix}{uuid.uuid4().hex[:12]}"
# ---------------------------------------------------------------------
# Agent Runtime
# ---------------------------------------------------------------------
class AgentRuntime:
    """确定性 Agent 运行内核。
    用法：
        runtime = AgentRuntime()                        # 全部默认依赖
        result = runtime.run(AgentRunRequest(...))      # 同步返回
        runtime.resume(run_id) / runtime.cancel(...)
    """
    def __init__(
        self,
        *,
        tool_registry: ToolRegistry | None = None,
        policy_guard: PolicyGuard | None = None,
        memory_store: MemoryStore | None = None,
        run_repository: RunRepository | None = None,
        approval_repository: ApprovalRepository | None = None,
        clock: Clock | None = None,
        id_factory: IdFactory | None = None,
        runtime_config: RuntimeConfig | None = None,
        model_adapter: ModelAdapterPlanner | None = None,
        lesson_store: LessonStore | None = None,
    ) -> None:
        self.config = runtime_config or RuntimeConfig()
        self.clock: Clock = clock or SystemClock()
        self.id_factory: IdFactory = id_factory or UuidIdFactory()
        self.registry = tool_registry or ToolRegistry()
        self.policy_guard: PolicyGuard = policy_guard or RulePolicyGuard(self.config)
        self.memory: MemoryStore = memory_store or InMemoryMemoryStore(
            id_factory=self.id_factory, clock=self.clock
        )
```

```
        # 跨 run 经验：与 run 级 working 记忆共用同一个 MemoryStore，


        # 于是「经验」与「记忆」天然遵循同一套脱敏与留痕纪律。
        self.lessons: LessonStore = lesson_store or LessonStore(
            self.memory,
            enabled=self.config.lessons_enabled,
            id_factory=self.id_factory,
            clock=self.clock,
        )
        self.runs: RunRepository = run_repository or InMemoryRunRepository()
        self.approvals: ApprovalRepository = approval_repository or InMemoryApprovalRepository()
        self.executor = ToolExecutor(self.registry, self.policy_guard, self.clock)
        self._planner = RulePlanner(self.config)
        self.model_adapter = model_adapter
        self._boot_ms = self.clock.now_ms()
    # ------------------------------------------------------------------
    # 公共入口（手册 3.6）
    # ------------------------------------------------------------------
    def run(self, request: AgentRunRequest) -> AgentRunResult:
        self._sweep_expired()
        # 重复触发幂等：同一幂等键复用已有 run，不新建
        if request.idempotency_key:
            existing = self.runs.find_by_idempotency_key(request.idempotency_key)
            if existing is not None:
                logger.info("[Agent] 幂等触发复用 run %s (key=%s) ", existing.run_id, request.idemp…
                return self._result(existing, idempotent_replay=True)
        now = self.clock.now()
        run = AgentRun(
            run_id=self.id_factory.new("run_"),
            trigger_type=request.trigger_type,
            objective=request.objective,
            status=AgentStatus.CREATED.value,
            policy_version=None,
            started_at=now,
            finished_at=None,
            termination_reason=None,
            trace_id=request.trace_id or uuid.uuid4().hex,
            created_at=now,
            updated_at=now,
            request=request,
            runtime_state={"stage": "created"},
        )
        self.runs.create(run)
        return self._advance(run)
    def resume(self, run_id: str) -> AgentRunResult:
        run = self.runs.get(run_id)
        if run is None:
            raise AgentNotFoundError(f"run 不存在：{run_id}")
        if run.status in TERMINAL_STATUSES:
            return self._result(run)  # 终态不得再次迁移
         self._sweep_expired()
```

```
        refreshed = self.runs.get(run_id)

        if refreshed.status in TERMINAL_STATUSES:
            return self._result(refreshed)
        return self._advance(refreshed)
    def cancel(self, run_id: str, actor: str, reason: str) -> AgentRunResult:
        run = self.runs.get(run_id)
        if run is None:
            raise AgentNotFoundError(f"run 不存在：{run_id}")
        if run.status in TERMINAL_STATUSES:
            return self._result(run)  # 终态不可再迁移，cancel 为幂等 no-op
        # 挂起中的审批随取消关闭
        for approval_id in run.runtime_state.get("approvals") or []:
            record = self.approvals.get(approval_id)
            if record is not None and record.decision is None:
                self.approvals.decide(approval_id, actor, "cancelled", reason or "run 被取消")
        self._add_terminal_step(
            run,
            status=AgentStatus.CANCELLED.value,
            decision_summary=f"run 被 {actor} 取消：{reason}",
            error_code=None,
        )
        run.status = AgentStatus.CANCELLED.value
        run.termination_reason = AgentStatus.CANCELLED.value
        run.finished_at = self.clock.now()
        run.updated_at = self.clock.now()
        self.runs.save(run)
        return self._result(run)
    def status(self) -> RuntimeStatus:
        runs = self.runs.list()
        active = [r for r in runs if r.status not in TERMINAL_STATUSES]
        if self.model_adapter is None:
            model_available = self.config.model_available
        else:
            model_available = bool(
                self.model_adapter.enabled and self.model_adapter.client is not None
            )
        return RuntimeStatus(
            state="running" if active else "idle",
            model_available=model_available,
            rule_mode=not model_available,
            policy_version=self.config.policy_version,
            active_runs=len(active),
            total_runs=len(runs),
            pending_approvals=len(self.approvals.list_pending()),
            tools_registered=len(self.registry),
            uptime_ms=self.clock.now_ms() - self._boot_ms,
        )
    # ------------------------------------------------------------------
    # 状态机推进（run 与 resume 共用）
    # ------------------------------------------------------------------
```

```
    def _advance(self, run: AgentRun) -> AgentRunResult:

        while True:
            stage = run.runtime_state.get("stage", "created")
            if stage == "created":
                result = self._stage_planning(run)
            elif stage in ("planning", "waiting_policy"):
                # 阶段名与状态对齐：planning 阶段产出计划并置 waiting_policy；
                # waiting_policy 阶段执行策略守卫（replan 也会回到 planning）
                result = self._stage_policy(run)
            elif stage == "waiting_approval":
                outcome = self._resolve_approvals(run)
                if outcome == "pending":
                    return self._result(run)  # 审批未决，保持挂起
                if outcome == "rejected":
                    return self._fail(run, ErrorCode.APPROVAL_REJECTED, "人工审批拒绝", verify=Fals…
                if outcome == "timeout":
                    return self._fail(run, ErrorCode.APPROVAL_TIMEOUT, "人工审批超时", verify=False)
                run.runtime_state["stage"] = "executing"
                result = None
            elif stage == "executing":
                result = self._stage_execute(run)
            elif stage == "verifying":
                result = self._succeed(run)
            else:
                result = self._fail(run, ErrorCode.INTERNAL_ERROR, f"未知阶段：{stage}", verify=Fal…
            if isinstance(result, AgentRunResult):
                return result
            # result is None -> 继续推进
    def _stage_planning(self, run: AgentRun) -> None:
        self._transition(run, AgentStatus.PLANNING)
        try:
            plan, source, error_code = self._build_plan(run)
        except AgentError as exc:
            return self._fail(run, ErrorCode.INTERNAL_ERROR, f"规划失败：{exc}", verify=False)
        if not plan:
            return self._fail(run, ErrorCode.INTERNAL_ERROR, "规划结果为空", verify=False)
        run.runtime_state["plan"] = plan
        run.runtime_state["plan_idx"] = 0
        run.runtime_state["bindings"] = dict(run.request.params or {})
        run.runtime_state["replan_count"] = 0
        # 每个规划周期落一条 plan 步骤（决策摘要枚举全部动作；重规划时
        # 新增 replan + 新的 plan 步骤，不覆盖历史）
        plan_summary = self._plan_summary(plan, source=source, error_code=error_code)
        role_summary = self._describe_plan_roles(plan)
        if role_summary:
            plan_summary = f"{plan_summary}；{role_summary}"
        lesson_note = self._consult_lessons(run)
        if lesson_note:
            plan_summary = f"{plan_summary}；{lesson_note}"
        if not self._add_step(
```

```
            run,
        step_type=AgentStepType.PLAN,
        decision_summary=plan_summary,
    ):
        return self._fail(run, ErrorCode.MAX_STEPS_EXCEEDED, "步骤预算耗尽（计划）", verify=Fal…
    run.runtime_state["stage"] = "waiting_policy"
    return None
def _consult_lessons(self, run: AgentRun) -> str:
    """规划阶段检索跨 run 经验，命中则返回写进计划摘要的那句话。
    为什么不做「经验直接改计划」：那会让同一条经验在不同版本间
      产生难以复现的行为漂移，而且评审核对时无法解释"这一步为什么
      和上次不一样"。这里只把经验写进决策摘要（给人和审计看），
      计划本身仍由确定性规则生成：经验影响的是"解释"与"优先级提示"，
      不是隐藏地改写控制流。
    """
    main_class = str(run.request.params.get("main_class") or "")
    if not main_class:
        return ""
    hits = self.lessons.match(
        main_class=main_class,
        context=dict(run.request.lessons_context or {}),
    )
    if not hits:
        return ""
    self.lessons.note_hit(hits, run_id=run.run_id)
    run.runtime_state["lesson_hits"] = [lesson.lesson_id for lesson in hits]
    return summarize_hits(hits)
def _build_plan(self, run: AgentRun) -> tuple[list[RuleStep], str | None, str | None]:
    """生成一次计划；适配器启用时由适配器负责模型优先与规则兜底。"""
    if self.model_adapter is None:
        return self._planner.build(run.request), None, None
    proposal = self.model_adapter.plan(
        task=run.request,
        business_context=dict(run.runtime_state.get("business_context") or {}),
        tool_summary=summarize_tools(self.registry),
        role=run.request.role,
    )
    return list(proposal.steps), proposal.source, proposal.error_code
@staticmethod
def _plan_summary(
    plan: list[RuleStep],
    *,
    source: str | None = None,
    error_code: str | None = None,
) -> str:
    actions = "; ".join(f"{s.tool} ({s.summary or '无摘要'}) " for s in plan)
    if source is None:
        return f"规则计划 {len(plan)} 步：{actions}"
    fallback = f" code={error_code}" if error_code else ""
    return f"规划 source={source}{fallback}，{len(plan)} 步：{actions}"
```

```
    @staticmethod

    def _describe_plan_roles(plan: list[RuleStep]) -> str:
        """多角色计划的人话摘要：按角色分组列出动作。
        单角色计划（全部 role 为 None）返回空串，调用方保持原有摘要文案不变——
        评测与既有断言对 `_plan_summary` 的输出是敏感的，不能顺手改掉。
        """
        if not any(step.role for step in plan):
            return ""
        groups: list[tuple[str, list[RuleStep]]] = []
        for step in plan:
            role = step.role or "编排"
            if groups and groups[-1][0] == role:
                groups[-1][1].append(step)
            else:
                groups.append((role, [step]))
        parts = [
            "【{role}】{actions}".format(
                role=role,
                actions="; ".join(
                    f"{s.tool} ({s.summary or '无摘要'}) " for s in steps
                ),
            )
            for role, steps in groups
        ]
        return "角色分工：" + " → ".join(parts)
    def _stage_policy(self, run: AgentRun) -> None:
        self._transition(run, AgentStatus.WAITING_POLICY)
        plan: list[RuleStep] = run.runtime_state["plan"]
        decision = self.policy_guard.check_plan(plan, run.request, self.registry)
        run.policy_version = decision.policy_version or self.config.policy_version
        if not self._add_step(
            run,
            step_type=AgentStepType.POLICY,
            status="ok" if decision.allow else "failed",
            decision_summary=f"策略守卫 ({run.policy_version}) ：{decision.reason}",
            error_code=None if decision.allow else ErrorCode.POLICY_DENIED,
        ):
            return self._fail(run, ErrorCode.MAX_STEPS_EXCEEDED, "步骤预算耗尽（策略）", verify=Fal…
        if not decision.allow:
            return self._fail(run, ErrorCode.POLICY_DENIED, decision.reason, verify=False)
        # 审批预检：对需要审批的计划步骤一次性创建审批单并挂起
        approval_ids: list[str] = []
        for step in plan:
            tool = self.registry.get(step.tool)
            if tool is None:
                continue  # 执行阶段会以 tool_failed 失败
            tool_decision = self.policy_guard.check_tool(tool, run.request.role, {})
            if tool_decision.requires_approval:
                record = self.approvals.create(
                    ApprovalRecord(
```

```
                        approval_id=self.id_factory.new("apr_"),
                        run_id=run.run_id,
                        requested_action=f"{step.tool}: {step.summary}",
                        risk_level=(
                            tool_decision.risk_level.value
                            if tool_decision.risk_level is not None
                            else str(tool.risk_level)
                        ),
                        requested_by=run.request.actor,
                        requested_at=self.clock.now(),
                    )
                )
                if not self._add_step(
                    run,
                    step_type=AgentStepType.APPROVAL_REQUEST,
                    status="pending",
                    decision_summary=f"请求人工审批: {record.requested_action} (风险 {record.risk_l…
                ):
                    return self._fail(run, ErrorCode.MAX_STEPS_EXCEEDED, "步骤预算耗尽 (审批) ", ve…
                approval_ids.append(record.approval_id)
        if approval_ids:
            run.runtime_state["approvals"] = approval_ids
            run.runtime_state["stage"] = "waiting_approval"
            self._transition(run, AgentStatus.WAITING_APPROVAL)
            return None
        run.runtime_state["stage"] = "executing"
        self._transition(run, AgentStatus.EXECUTING)
        return None
    def _stage_execute(self, run: AgentRun) -> AgentRunResult | None:
        plan: list[RuleStep] = run.runtime_state["plan"]
        bindings: dict[str, Any] = run.runtime_state.setdefault("bindings", {})
        idx = int(run.runtime_state.get("plan_idx", 0))
        role = run.request.role
        while idx < len(plan):
            step = plan[idx]
            tool = self.registry.get(step.tool)
            if tool is None:
                return self._fail(run, ErrorCode.TOOL_FAILED, f"工具未注册: {step.tool}")
            self._transition(run, AgentStatus.EXECUTING)
            try:
                input_ = self._planner.bind(step, bindings)
            except AgentError as exc:
                return self._fail(run, exc.code or ErrorCode.INVALID_TOOL_INPUT, str(exc))
            # 执行 (含重试上限: 每次尝试都落一条 tool_call 步骤)
            attempt = 0
            final: ToolResult | None = None
            while True:
                attempt += 1
                final = self._execute_once_step(run, tool, step, input_, role)
                if final.ok or attempt > self.config.retry_limit:
```

```
                    break


                if final.error_code not in (ErrorCode.TOOL_TIMEOUT, ErrorCode.TOOL_FAILED):
                    break  # 策略/输入/输出错误不可重试
            self._transition(run, AgentStatus.OBSERVING)
            summary = self._observe_summary(step, final)
            if not self._add_step(
                run,
                step_type=AgentStepType.OBSERVATION,
                decision_summary=summary,
                role=step.role,
            ):
                return self._fail(run, ErrorCode.MAX_STEPS_EXCEEDED, "步骤预算耗尽（观察）")
            self._save_memory(run, source_type="tool_call", summary=summary)
            if not final.ok:
                code = final.error_code or ErrorCode.TOOL_FAILED
                if step.on_error == "replan":
                    if self._replan(run, code):
                        plan = run.runtime_state["plan"]
                        idx = int(run.runtime_state.get("plan_idx", 0))
                        continue
                    return self._fail(run, ErrorCode.MAX_STEPS_EXCEEDED, f"重规划预算耗尽（{code}）…
                return self._fail(run, code, final.error_message or f"步骤 {step.tool} 执行失败")
            if step.expect is not None:
                verdict, expect_code = self._check_expectation(step.expect, final.data)
                if verdict == "replan":
                    if self._replan(run, expect_code):
                        plan = run.runtime_state["plan"]
                        idx = int(run.runtime_state.get("plan_idx", 0))
                        continue
                    return self._fail(run, ErrorCode.MAX_STEPS_EXCEEDED, f"重规划预算耗尽（{expect_…
                if verdict == "terminate":
                    return self._fail(
                        run, expect_code, f"步骤 {step.tool} 未满足期望（{step.expect.key}）"
                    )
            if isinstance(final.data, dict):
                bindings.update(final.data)
            idx += 1
            run.runtime_state["plan_idx"] = idx
        run.runtime_state["stage"] = "verifying"
        return None
    # ------------------------------------------------------------------
    # 步骤记录
    # ------------------------------------------------------------------
    def _add_step(
        self,
        run: AgentRun,
        *,
        step_type: AgentStepType,
        decision_summary: str,
        status: str = "ok",
```

```
        tool_name: str | None = None,
        tool_version: str | None = None,
        input_hash: str | None = None,
        output_hash: str | None = None,
        latency_ms: int = 0,
        error_code: str | None = None,
        role: str | None = None,
    ) -> AgentStep | None:
        step_no = len(self.runs.steps(run.run_id)) + 1
        if step_no > self.config.max_steps:
            return None  # 步骤预算耗尽 → 安全终止
        step = AgentStep(
            step_id=self.id_factory.new("stp_"),
            run_id=run.run_id,
            step_no=step_no,
            step_type=step_type.value,
            decision_summary=decision_summary,
            tool_name=tool_name,
            tool_version=tool_version,
            input_hash=input_hash,
            output_hash=output_hash,
            status=status,
            latency_ms=latency_ms,
            error_code=error_code,
            created_at=self.clock.now(),
        )
        self.runs.add_step(step)
        self._record_step_role(run, step_no, role)
        return step
    @staticmethod
    def _record_step_role(run: AgentRun, step_no: int, role: str | None) -> None:
        """把「这一步是哪个角色做的」记进 run 状态快照。
        刻意不写进 t_agent_step：手册 3.7 的步骤表是冻结契约，角色归属
          是内核的运行期元数据。存在 runtime_state 里，既能跟随
          `persistent_repository` 的状态快照一起持久化（重启后只读回放
          仍有角色），又不需要任何 DDL。
        """
        if not role:
            return
        roles = run.runtime_state.setdefault("step_roles", {})
        if isinstance(roles, dict):
            roles[str(step_no)] = role
    def _add_terminal_step(
        self,
        run: AgentRun,
        *,
        status: str,
        decision_summary: str,
        error_code: str | None,
    ) -> None:
```

```
        """终态步骤必须记录，不受 max_steps 限制。"""

        step = AgentStep(
            step_id=self.id_factory.new("stp_"),
            run_id=run.run_id,
            step_no=len(self.runs.steps(run.run_id)) + 1,
            step_type=AgentStepType.TERMINAL.value,
            decision_summary=decision_summary,
            status=status,
            error_code=error_code if error_code in ERROR_CODES else None,
            created_at=self.clock.now(),
        )
        self.runs.add_step(step)
    def _execute_once_step(
        self,
        run: AgentRun,
        tool: ToolDefinition,
        step: RuleStep,
        input_: dict[str, Any],
        role: str,
    ) -> ToolResult:
        result = self.executor.execute_once(run.run_id, role, tool.name, input_)
        in_hash = sha256_hex(input_)
        out_hash = result.output_hash
        if out_hash is None:
            out_hash = (
                sha256_hex({"error": result.error_code, "message": result.error_message})
                if not result.ok
                else None
            )
        # 期望校验并入工具步骤状态：工具本身执行成功但业务期望未达成时，
        #   该条 tool_call 步骤必须落 failed（error_code=期望错误码），
        #   审计不允许静默丢弃（观察→验证→重规划的失败必须可追溯）。
        expect_ok = True
        expect_code: str | None = None
        if result.ok and step.expect is not None:
            verdict, expect_code = self._check_expectation(step.expect, result.data)
            expect_ok = verdict == "ok"
        step_ok = result.ok and expect_ok
        step_error_code = (
            result.error_code if not result.ok else (expect_code if not expect_ok else None)
        )
        self._add_step(
            run,
            step_type=AgentStepType.TOOL_CALL,
            status="ok" if step_ok else "failed",
            decision_summary=(
                f"调用 {tool.name} v{tool.version}：{step.summary or step.tool}"
                + ("（幂等重放）" if result.replayed else "")
                + ("" if step_ok else f"（期望未达成：{expect_code}）")
            ),
```

```
            tool_name=tool.name,
            tool_version=tool.version,
            input_hash=in_hash,
            output_hash=out_hash,
            latency_ms=result.latency_ms,
            error_code=step_error_code,
            role=step.role,
        )
        return result
    # ------------------------------------------------------------------
    # 验证 / 重规划 / 终止
    # ------------------------------------------------------------------
    def _check_expectation(self, expect: Any, data: dict[str, Any] | None) -> tuple[str, str]:
        """返回 (verdict, error_code)；verdict ∈ {"ok","terminate","replan"}。"""
        data = data or {}
        verdict = "terminate" if expect.on_violation == "terminate" else "replan"
        if expect.key not in data:
            return verdict, expect.error_code
        value = data[expect.key]
        if expect.min_items is not None and isinstance(value, (list, tuple, dict, str)):
            if len(value) < expect.min_items:
                return verdict, expect.error_code
        if expect.contains and value not in expect.contains:
            return verdict, expect.error_code
        return "ok", expect.error_code
    def _replan(self, run: AgentRun, reason_code: str) -> bool:
        """观察→验证→重规划：追加 verification(failed) + replan + 新 plan 步骤。
        只追加、不覆盖历史步骤（手册 3.2）；预算不足返回 False，
          由调用方安全终止（max_steps_exceeded 或原错误码）。
        """
        replan_count = int(run.runtime_state.get("replan_count", 0))
        if replan_count >= self.config.max_replans:
            return False
        if run.status != AgentStatus.VERIFYING.value:
            self._transition(run, AgentStatus.VERIFYING)
        if not self._add_step(
            run,
            step_type=AgentStepType.VERIFICATION,
            status="failed",
            decision_summary=f"重规划前验证失败：{reason_code}",
            error_code=reason_code,
        ):
            return False
        self._transition(run, AgentStatus.PLANNING)
        if not self._add_step(
            run,
            step_type=AgentStepType.REPLAN,
            decision_summary=f"第 {replan_count + 1} 次重规划（原因 {reason_code}）",
        ):
            return False
```

```
        try:
            new_plan, source, error_code = self._build_plan(run)
        except AgentError:
            return False
        if not new_plan:
            return False
        replan_summary = self._plan_summary(new_plan, source=source, error_code=error_code)
        replan_roles = self._describe_plan_roles(new_plan)
        if replan_roles:
            replan_summary = f"{replan_summary}; {replan_roles}"
        if not self._add_step(
            run,
            step_type=AgentStepType.PLAN,
            decision_summary=replan_summary,
        ):
            return False
        run.runtime_state["plan"] = new_plan
        run.runtime_state["plan_idx"] = 0
        run.runtime_state["replan_count"] = replan_count + 1
        return True
    def _terminate(
        self,
        run: AgentRun,
        status: AgentStatus,
        reason: str,
        message: str,
        *,
        verify: bool,
    ) -> AgentRunResult:
        if verify:
            if run.status != AgentStatus.VERIFYING.value:
                self._transition(run, AgentStatus.VERIFYING)
            ok = status == AgentStatus.SUCCEEDED
            self._add_step(
                run,
                step_type=AgentStepType.VERIFICATION,
                status="ok" if ok else "failed",
                decision_summary="所有计划步骤验证通过" if ok else f"验证失败：{message}",
                error_code=None if ok else (reason if reason in ERROR_CODES else None),
            )
        self._add_terminal_step(
            run,
            status=status.value,
            decision_summary=message,
            error_code=reason if reason in ERROR_CODES else None,
        )
        run.status = status.value
        run.termination_reason = reason
        run.finished_at = self.clock.now()
        run.updated_at = self.clock.now()
```

```
    def random_location(self, rng: random.Random) -> tuple[float, float]:
        """事件落点（带小幅漂移，模拟垃圾随潮流移动）。"""
        if self.patrol_radius_deg > 0:
            # 无人机/广域设备：巡飞范围内随机
            return (
                self.lng + rng.uniform(-self.patrol_radius_deg, self.patrol_radius_deg),
                self.lat + rng.uniform(-self.patrol_radius_deg, self.patrol_radius_deg),
            )
        # 固定摄像头：画面覆盖范围内的近岸水域（约 ±0.004 度 ≈ 400m）
        return (
            self.lng + rng.uniform(-0.004, 0.004),
            self.lat + rng.uniform(-0.004, 0.004),
        )
    def pick_class(self, rng: random.Random) -> str:
        """按该点位的垃圾构成抽取类别。"""
        classes = list(self.profile.keys())
        weights = list(self.profile.values())
        return rng.choices(classes, weights=weights, k=1)[0]
    def next_seq(self) -> int:
        self.seq += 1
        return self.seq
def _make_bbox(cls: str, rng: random.Random) -> list[int]:
    """生成该类别尺度合理的检测框。"""
    visual = CLASS_VISUAL[cls]
    lo, hi = visual["size"]
    w = rng.randint(lo, hi)
    h = int(w * rng.uniform(0.6, 1.2))
    x1 = rng.randint(0, max(1, FRAME_W - w - 1))
    y1 = rng.randint(0, max(1, FRAME_H - h - 1))
    return [x1, y1, x1 + w, y1 + h]
def _make_confidence(cls: str, rng: random.Random) -> float:
    lo, hi = CLASS_VISUAL[cls]["conf"]
    return round(rng.uniform(lo, hi), 3)
def generate_raw_frame(device: SimDevice, rng: random.Random) -> list[dict[str, Any]]:
    """生成一帧的原始检测结果（未经时序校验）。
    这里会故意混入「瞬态噪声」—— 模拟浪花反光、飞鸟、光斑。
    真实边缘盒的原始输出就长这样：大部分是噪声，靠时序校验筛。
    两种检测的区别体现在连续性上：
    - 持续目标：位于少量固定区域（每台设备维护若干个"垃圾位置"），
      逐帧微小漂移，因此能被跟踪器连续命中
    - 瞬态噪声：每次随机落点，前后帧对不上，被跟踪器自然饿死
    """
    dets: list[dict[str, Any]] = []
    # 持续目标（会被时序校验确认）
    # 每台设备有 2~4 个固定的「垃圾聚集位置」，模拟水面漂浮物
    if not hasattr(device, "_litter_spots"):
        device._litter_spots = [  # type: ignore[attr-defined]
            (
                rng.randint(100, FRAME_W - 300),
                rng.randint(80, FRAME_H - 200),
```

```
                device.pick_class(rng),
            )
            for _ in range(rng.randint(2, 4))
        ]
    for spot_x, spot_y, cls in device._litter_spots:  # type: ignore[attr-defined]
        if rng.random() < 0.78:      # 每帧较大概率出现 → 能连续命中
            w = rng.randint(30, 90)
            h = int(w * rng.uniform(0.7, 1.1))
            # 逐帧微小漂移（水流推动），位移 < match_distance 才能被匹配上
            jitter_x = rng.randint(-14, 14)
            jitter_y = rng.randint(-10, 10)
            x1 = max(0, min(FRAME_W - w - 1, spot_x + jitter_x))
            y1 = max(0, min(FRAME_H - h - 1, spot_y + jitter_y))
            dets.append(
                {
                    "class": cls,
                    "confidence": _make_confidence(cls, rng),
                    "bbox": [x1, y1, x1 + w, y1 + h],
                }
            )
    # 瞬态噪声（不会通过时序校验，被跟踪器饿死）
    if rng.random() < 0.50:
        for _ in range(rng.randint(1, 3)):
            cls = rng.choice(WASTE_CLASSES)
            dets.append(
                {
                    "class": cls,
                    # 噪声置信度偏低，进一步降低通过率
                    "confidence": round(rng.uniform(0.30, 0.62), 3),
                    "bbox": _make_bbox(cls, rng),
                }
            )
    return dets
# ---------------------------------------------------------------------
# 模拟器主体
# ---------------------------------------------------------------------
class EdgeSimulator:
    """边缘盒模拟器。"""
    def __init__(self, args: argparse.Namespace, config: dict[str, Any]) -> None:
        self.args = args
        self.config = config
        self.rng = random.Random(args.seed)
        self.site_id = config.get("site_id", "lianjiang")
        self.prefix = "marine"
        self.mqtt_conf = config.get("mqtt", {})
        self.temporal_conf = config.get("temporal", {})
        self.reporting_conf = config.get("reporting", {})
        self.buffer_conf = config.get("buffer", {})
        self.inference_conf = config.get("inference", {})
        self.devices = self._load_devices()
```

```
        self.buffer = OutboxBuffer(max_size=int(self.buffer_conf.get("max_size", 1000)))



        # 统计
        self.counter = {
            "raw_frames": 0,
            "raw_detections": 0,
            "confirmed_events": 0,
            "suppressed_noise": 0,
            "published": 0,
            "duplicates": 0,
            "buffered": 0,
            "flushed": 0,
            "net_drops": 0,
        }
        self._stop = False
        self._connected = False
        self._client: mqtt.Client | None = None
    # 初始化
    def _load_devices(self) -> list[SimDevice]:
        devices: list[SimDevice] = []
        wanted = None
        if self.args.devices:
            wanted = {d.strip() for d in self.args.devices.split(",") if d.strip()}
        for raw in self.config.get("devices", []):
            if wanted and raw["device_id"] not in wanted:
                continue
            profile = raw.get("profile", {})
            if not profile:
                # 未配置构成时给个通用分布
                profile = {c: 0.25 for c in WASTE_CLASSES}
            dev = SimDevice(
                device_id=raw["device_id"],
                name=raw.get("name", raw["device_id"]),
                device_type=raw.get("device_type", "shore_camera"),
                lng=float(raw["lng"]),
                lat=float(raw["lat"]),
                profile=profile,
                rate=float(raw.get("rate", 1.0)),
                patrol_radius_deg=float(raw.get("patrol_radius_deg", 0.0)),
            )
            if self.temporal_conf.get("enabled", True):
                dev.validator = TemporalValidator(
                    window_frames=int(self.temporal_conf.get("window_frames", 15)),
                    min_hits=int(self.temporal_conf.get("min_hits", 3)),
                    grid_size=int(self.temporal_conf.get("grid_size", 64)),
                    min_confidence=float(self.temporal_conf.get("min_confidence", 0.45)),
                    # 下面三个必须显式传：漏传会静默落到类默认值，
                    #   于是 config.yaml 里改了也不生效：排查起来极难。
                    match_distance=float(self.temporal_conf.get("match_distance", 60.0)),
                    max_misses=int(self.temporal_conf.get("max_misses", 5)),
                    min_iou=float(self.temporal_conf.get("min_iou", 0.25)),
```

```
                decimation=int(self.temporal_conf.get("decimation", 4)),
            )
            devices.append(dev)
        # 恢复 seq（崩溃重启后从上次值续上，保证幂等判断有效）
        state = self._load_state()
        for dev in devices:
            if dev.device_id in state:
                dev.seq = int(state[dev.device_id])
        return devices
    def _state_path(self) -> Path:
        return Path(__file__).with_name(STREAM_STATE_FILE)
    def _load_state(self) -> dict[str, int]:
        path = self._state_path()
        if not path.exists():
            return {}
        try:
            return json.loads(path.read_text(encoding="utf-8"))
        except Exception:   # noqa: BLE001
            return {}
    def _save_state(self) -> None:
        try:
            data = {d.device_id: d.seq for d in self.devices}
            self._state_path().write_text(
                json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8"
            )
        except Exception:   # noqa: BLE001
            pass
    # MQTT
    def _new_client(self) -> mqtt.Client:
        """创建 MQTT 客户端。
        注意 paho-mqtt 2.x 的 callback_api_version 变更——
        传错会直接抛 ValueError，这是最常见的踩坑点。
        """
        try:
            client = mqtt.Client(
                mqtt.CallbackAPIVersion.VERSION2,
                client_id=f"edge-sim-{uuid.uuid4().hex[:8]}",
                protocol=mqtt.MQTTv5,
            )
        except (AttributeError, ValueError):
            # 回退到 1.x API
            client = mqtt.Client(client_id=f"edge-sim-{uuid.uuid4().hex[:8]}")
        if self.mqtt_conf.get("username"):
            client.username_pw_set(
                self.mqtt_conf["username"], self.mqtt_conf.get("password", "")
            )
        # 遗嘱消息：异常断开时 Broker 代发，平台据此判定离线
        if self.mqtt_conf.get("use_lwt", True):
            for dev in self.devices:
                pass  # LWT 每设备一条，见下面 _register_lwt
```

```
            self._lwt_topic = f"{self.prefix}/{self.site_id}/+/{'status'}"



        client.on_connect = self._on_connect
        client.on_disconnect = self._on_disconnect
        client.on_publish = self._on_publish
        return client
    def _on_connect(self, client, userdata, flags, reason_code, properties=None) -> None:
        self._connected = True
        print(f"[MQTT] 已连接 Broker {self.args.host}:{self.args.port}")
        # 上线后立即：1) 宣告在线 2) 补传断网期间积压的事件
        for dev in self.devices:
            self._publish_status(dev, online=True)
        if len(self.buffer) > 0:
            print(f"[补传] 恢复连接，buffer 中有 {len(self.buffer)} 条待补发")
            sent = self.flush_buffer()
            print(f"[补传] 本轮补发 {sent} 条")
    def _on_disconnect(self, client, userdata, *args) -> None:
        self._connected = False
        print("[MQTT] 连接已断开，事件将进入本地缓存")
    def _on_publish(self, *args) -> None:
        pass
    # 上报
    def _topic_event(self, dev: SimDevice) -> str:
        return f"{self.prefix}/{self.site_id}/{dev.device_id}/event"
    def _topic_telemetry(self, dev: SimDevice) -> str:
        return f"{self.prefix}/{self.site_id}/{dev.device_id}/telemetry"
    def _topic_status(self, dev: SimDevice) -> str:
        return f"{self.prefix}/{self.site_id}/{dev.device_id}/status"
    def _publish(self, topic: str, payload: dict[str, Any], qos: int) -> None:
        """发布消息。未连上或模拟断网时进本地缓存。"""
        body = json.dumps(payload, ensure_ascii=False)
        if self.args.dry_run:
            print(f"[DRY-RUN] {topic} qos={qos}\n  {body[:200]}")
            self.counter["published"] += 1
            return
        if not self._connected:
            if self.buffer_conf.get("enabled", True):
                self.buffer.put(topic, payload, qos)
                self.counter["buffered"] += 1
            return
        try:
            result = self._client.publish(topic, body, qos=qos, retain=False)
            if getattr(result, "rc", 0) != 0:
                # 发布队列满等异常 → 缓存待补
                if self.buffer_conf.get("enabled", True):
                    self.buffer.put(topic, payload, qos)
                    self.counter["buffered"] += 1
            else:
                self.counter["published"] += 1
        except Exception as exc:   # noqa: BLE001
            print(f"[警告] 发布失败：{exc}（已缓存待补传）")
```

```
            self.buffer.put(topic, payload, qos)



            self.counter["buffered"] += 1
    def _publish_status(self, dev: SimDevice, online: bool) -> None:
        """设备上下线状态 (retained) 。"""
        payload = {
            "device_id": dev.device_id,
            "online": online,
            "ts": _now_iso(),
        }
        if self.args.dry_run or self._client:
            self._publish(self._topic_status(dev), payload, qos=1)
    def _publish_telemetry(self, dev: SimDevice) -> None:
        """心跳遥测。QoS0：丢一帧无所谓。"""
        payload = {
            "device_id": dev.device_id,
            "device_type": dev.device_type,
            "timestamp": _now_iso(),
            "status": "online",
            "battery": dev.battery,
            "location": {"lng": dev.lng, "lat": dev.lat},
        }
        self._publish(self._topic_telemetry(dev), payload, qos=0)
    def build_event(self, dev: SimDevice, det: dict[str, Any]) -> dict[str, Any]:
        """构造符合平台契约的事件报文。"""
        seq = dev.next_seq()
        lng, lat = dev.random_location(self.rng)
        cls = det["class"]
        return {
            "event_id": f"evt_{dev.device_id}_{seq:06d}",
            "device_id": dev.device_id,
            "device_type": dev.device_type,
            "timestamp": _now_iso(),
            "location": {"lng": round(lng, 6), "lat": round(lat, 6)},
            "detections": [det],
            "aggregate": {
                "main_class": cls,
                "count": 1,
                "max_confidence": det["confidence"],
            },
            "evidence_url": None,
            "model_version": self.inference_conf.get("model_version", "det_v0.1.0"),
            "seq": seq,
        }
    # 主循环
    def _tick_device(self, dev: SimDevice) -> list[dict[str, Any]]:
        """跑一台设备的一帧：生成原始检测 → 时序校验 → 产出确认事件。"""
        raw = generate_raw_frame(dev, self.rng)
        self.counter["raw_frames"] += 1
        self.counter["raw_detections"] += len(raw)
        if dev.validator is None:
```

```
            # 未启用时序校验：直接放行置信度达标的检测


            threshold = float(self.inference_conf.get("confidence_threshold", 0.45))
            return [d for d in raw if d["confidence"] >= threshold]
        confirmed = dev.validator.push(raw)
        # 抽帧推理下，miss 衰减必须按「被跳过的帧」推进，不能按 push 次数
        dev.validator.sweep(dev.validator.decimation)
        self.counter["confirmed_events"] += len(confirmed)
        return confirmed
    def _cooldown_ok(self, dev: SimDevice, det: dict[str, Any]) -> bool:
        """上报节流：同一格网冷却期内不重复上报。
        没有这一步，一块持续存在的泡沫会被连续上报几十次，
        平台的派单防抖虽然有 10 分钟合并窗口，但事件表会被灌爆。
        --event-interval 可覆盖配置中的冷却值，用来控制演示节奏
        （真实边缘盒用 90 秒；演示时想快点看到事件可设成 3~5 秒）。
        """
        if self.args.event_interval is not None:
            cooldown = float(self.args.event_interval)
        else:
            cooldown = float(self.reporting_conf.get("cooldown_seconds", 90))
        if cooldown <= 0:
            return True
        if dev.validator is not None:
            cell = dev.validator._cell(det["bbox"])
        else:
            cell = (0, 0)
        now = time.time()
        last = dev._last_report.get(cell, 0.0)
        if now - last < cooldown:
            return False
        dev._last_report[cell] = now
        return True
    def flush_buffer(self) -> int:
        """补传本地缓存的事件。"""
        batch = int(self.buffer_conf.get("flush_batch", 20))
        def _send(topic: str, payload: str, qos: int) -> None:
            try:
                self._client.publish(topic, payload, qos=qos, retain=False)
            except Exception:   # noqa: BLE001
                pass
        sent = self.buffer.flush(_send, batch=batch)
        self.counter["flushed"] += sent
        return sent
    def run(self) -> None:
        """启动主循环。"""
        if self.args.dry_run:
            self._client = None
        else:
            self._client = self._new_client()
            try:
                self._client.connect(
```

```
                self.args.host,
                self.args.port,
                keepalive=int(self.mqtt_conf.get("keepalive", 60)),
            )
            self._client.loop_start()
        except Exception as exc:   # noqa: BLE001
            print(f"[错误] 无法连接 MQTT Broker {self.args.host}:{self.args.port}")
            print(f"        {exc}")
            print("        提示：先执行 docker compose up -d emqx，或用 --dry-run 查看报文")
            sys.exit(2)
        time.sleep(1)
    self._print_banner()
    started = time.time()
    tick = 0
    # 关键：帧间隔模拟摄像头的抽帧节奏，不是「事件上报间隔」。
    #   真实边缘盒抽帧到 ~8fps（每 3 帧取 1 帧），所以帧间隔约 0.125s。
    #   时序校验的 window_frames=15 帧 ≈ 1.9 秒，正好覆盖一个浪周期。
    #   若把帧间隔设成几秒（早期实现就是这样），15 帧要跑一分钟，
    #   窗口长年填不满，用户会以为「系统不工作」。
    frame_interval = max(0.01, float(self.args.interval))
    # 事件上报冷却：真正的演示节奏旋钮。
    # None 表示用配置文件里的 cooldown_seconds（真实边缘盒的行为）。
    cooldown_override = self.args.event_interval
    try:
        while not self._stop:
            tick += 1
            elapsed = time.time() - started
            if self.args.duration and elapsed >= self.args.duration:
                print(f"\n[完成] 已达到运行时长 {self.args.duration}s")
                break
            # 模拟网络抖动
            if (
                self.args.net_drop_every
                and tick % self.args.net_drop_every == 0
                and self._connected
            ):
                self._simulate_net_drop()
            for dev in self.devices:
                confirmed = self._tick_device(dev)
                # 速率倍率 > 1 时，一帧内重复抽检（模拟更密集的垃圾分布）
                extra_rounds = max(0, int(dev.rate) - 1)
                for _ in range(extra_rounds):
                    confirmed.extend(self._tick_device(dev))
                for det in confirmed:
                    if not self._cooldown_ok(dev, det):
                        continue
                    event = self.build_event(dev, det)
                    self._emit_event(dev, event)
                # 每 5 轮发一次遥测心跳
                if tick % 5 == 0:
```

```
                    self._publish_telemetry(dev)



                    dev.battery = max(5, dev.battery - 1)
            # buffer 有积压且已恢复连接 → 持续补传
            if len(self.buffer) > 0 and self._connected and not self.args.dry_run:
                self.flush_buffer()
            self._print_progress(tick)
            self._save_state()
            # 精确对齐帧节奏：扣除本轮实际耗时，避免漂移
            sleep_for = frame_interval - (time.time() - started - elapsed)
            if sleep_for > 0:
                time.sleep(sleep_for)
    except KeyboardInterrupt:
        print("\n[中断] 收到 Ctrl+C，正在退出...")
    finally:
        self._shutdown()
def _emit_event(self, dev: SimDevice, event: dict[str, Any]) -> None:
    """发布事件，并按 --dup-rate 概率重复投递（验证平台幂等）。"""
    topic = self._topic_event(dev)
    self._publish(topic, event, qos=1)
    # 故意重复投递：模拟 QoS1 在网络抖动下的重传
    dup_rate = float(self.args.dup_rate or 0)
    if dup_rate > 0 and self.rng.random() < dup_rate:
        self.counter["duplicates"] += 1
        self._publish(topic, event, qos=1)
    label = CLASS_LABELS.get(event["aggregate"]["main_class"], "")
    print(
        f"  ↑ {dev.device_id} seq={event['seq']:>5} "
        f"{label} conf={event['aggregate']['max_confidence']:.2f} "
        f"({event['location']['lng']:.4f},{event['location']['lat']:.4f})"
    )
def _simulate_net_drop(self) -> None:
    """模拟一次网络中断：断开 → N 秒后恢复。"""
    self.counter["net_drops"] += 1
    print(f"\n[网络] 模拟断网（第 {self.counter['net_drops']} 次），事件转入本地缓存...")
    try:
        self._client.disconnect()
    except Exception:   # noqa: BLE001
        pass
    self._connected = False
    time.sleep(2.0)
    print("[网络] 尝试重连...")
    try:
        self._client.reconnect()
        time.sleep(1.0)
    except Exception as exc:   # noqa: BLE001
        print(f"[网络] 重连失败：{exc}")
def _shutdown(self) -> None:
    self._save_state()
    if self._client is not None:
        try:
```

```
            # 优雅下线：发 retained 离线状态，让平台立刻感知


            for dev in self.devices:
                self._publish_status(dev, online=False)
            time.sleep(0.3)
            self._client.loop_stop()
            self._client.disconnect()
        except Exception:   # noqa: BLE001
            pass
    self._print_summary()
# 输出
def _print_banner(self) -> None:
    total_rate = sum(d.rate for d in self.devices)
    window = self.temporal_conf.get("window_frames", 15)
    fps = 1.0 / max(0.01, float(self.args.interval))
    cooldown = (
        float(self.args.event_interval)
        if self.args.event_interval is not None
        else float(self.reporting_conf.get("cooldown_seconds", 90))
    )
    print("=" * 66)
    print("  探海灵眸 Oceanus — 边缘感知模拟器")
    print("=" * 66)
    print(f"  站点     : {self.site_id}")
    print(f"  设备数    : {len(self.devices)} 台 (合计速率倍率 {total_rate:.1f}) ")
    print(f"  场景     : {self.args.scenario}")
    print(f"  时序校验   : {'启用' if self.temporal_conf.get('enabled', True) else '关闭'}"
          f" (窗口 {window} 帧 / 最少 {self.temporal_conf.get('min_hits', 3)} 次命中 "
          f"≈ {window / fps:.1f}s 填满) ")
    print(f"  抽帧节奏   : {self.args.interval:.3f}s/帧 (≈{fps:.1f} fps) ")
    print(f"  上报冷却   : {cooldown:.0f}s / 目标")
    print(f"  重复投递比例 : {self.args.dup_rate:.0%}")
    print(f"  断网模拟   : {'每 %d 轮一次' % self.args.net_drop_every if self.args.net_drop_e…
    if self.args.dry_run:
        print("  模式     : DRY-RUN (不连接 Broker，只打印报文) ")
    print("=" * 66)
    print("  设备清单：")
    for dev in self.devices:
        print(f"    · {dev.device_id:<18} {dev.name}  起始 seq={dev.seq}")
    print("=" * 66)
def _print_progress(self, tick: int) -> None:
    if tick % 10 != 0:
        return
    c = self.counter
    # 增量累加各设备的统计 (drain 会清零，避免重复计数)
    tracked = 0
    for dev in self.devices:
        if dev.validator is not None:
            st = dev.validator.drain_stats()
            c["suppressed_noise"] += st["suppressed"]
            c["raw_detections"] += st["fed"]
```

```
                tracked += st["tracked"]



        noise_ratio = (
            c["suppressed_noise"] / c["raw_detections"] if c["raw_detections"] else 0
        )
        print(
            f"  [统计] 轮次={tick} 原始检测={c['raw_detections']} "
            f"确认事件={c['confirmed_events']} 噪声过滤={noise_ratio:.0%} "
            f"跟踪中={tracked} 已发布={c['published']} 缓存={len(self.buffer)}"
        )
    def _print_summary(self) -> None:
        c = self.counter
        # 把各设备剩余的统计收干净，保证汇总数字完整。
        # 口径与 _print_progress 保持一致：fed / absorbed / suppressed 同单位（都是检测数）。
        tracked = 0
        for dev in self.devices:
            if dev.validator is not None:
                st = dev.validator.drain_stats()
                c["suppressed_noise"] += st["suppressed"]
                c["raw_detections"] += st["fed"]
                tracked += st["tracked"]
        print("\n" + "=" * 66)
        print("  运行统计")
        print("=" * 66)
        print(f"  原始帧数     : {c['raw_frames']}（7 台设备逐帧判定）")
        print(f"  原始检测数    : {c['raw_detections']}（含瞬态噪声）")
        print(f"  时序校验确认事件：{c['confirmed_events']}")
        print(f"  被挡下的误报   : {c['suppressed_noise']}")
        if c["raw_detections"]:
            print(f"  误报抑制率    : {c['suppressed_noise'] / c['raw_detections']:.1%}")
        print(f"  成功发布     : {c['published']}（含重复投递 {c['duplicates']}）")
        print(f"  本地缓存/补传  : {c['buffered']} / {c['flushed']}")
        print(f"  模拟断网次数   : {c['net_drops']}")
        print("=" * 66)
        print("  说明：误报抑制率 = 从未被稳定跟踪的检测数 / 喂进校验器的检测总数。")
        print("        三个计数（fed/absorbed/suppressed）同单位，因此抑制率恒在")
        print("        0~100%，且满足 fed = absorbed + suppressed。")
        print("        抑制来自两道闸：① 置信度门限直接丢弃；")
        print("        ② 抽帧推理下，只在单帧闪现的浪花几乎不会被连续命中，")
        print("           凑不满 min_hits 便被窗口自然淘汰。")
        print("        注意分母是「检测数」不是「帧数」—— 同一目标在连续帧里")
        print("        贡献多个检测，稳定目标因此能被反复吸收。")
        print("=" * 66)
        print("  下一步：打开 http://localhost:8000/docs 查看事件是否入库，")
        print("          或访问前端大屏看告警与工单。")
        print("=" * 66)
# ---------------------------------------------------------------------
# 场景
# ---------------------------------------------------------------------
def apply_scenario(args: argparse.Namespace, config: dict[str, Any]) -> dict[str, Any]:
    """按场景调整参数。
```

```
    注意 `--interval` 已改为「抽帧间隔」（默认 0.125s ≈ 8fps），


    场景预设不再改它：真实摄像头帧率不因演示场景而变。
    控制演示节奏的是 `--event-interval`（上报冷却）。
    """
    scenario = args.scenario
    if scenario == "stress":
        # 压测：放宽确认门槛 + 缩短冷却 + 提高设备速率，把派单引擎压到极限
        config["temporal"]["window_frames"] = 6
        config["temporal"]["min_hits"] = 2
        config["reporting"]["cooldown_seconds"] = 5
        config["buffer"]["max_size"] = 5000
        if args.event_interval is None:
            args.event_interval = 1.0
        for dev in config.get("devices", []):
            dev["rate"] = float(dev.get("rate", 1.0)) * 3
    elif scenario == "regression":
        # 回归：时序校验更严格、冷却更长，结果稳定可复现，用于冒烟比对
        config["temporal"]["window_frames"] = 20
        config["temporal"]["min_hits"] = 5
        config["reporting"]["cooldown_seconds"] = 180
        if args.event_interval is None:
            args.event_interval = 10.0
    elif scenario == "demo":
        # 演示：真实帧率 + 配置文件里的冷却（90s）。
        # 想让事件出得更快，显式传 --event-interval 3。
        pass
    else:
        print(f"[警告] 未知场景 {scenario}，按 demo 处理")
    return config
# ---------------------------------------------------------------------
# 工具
# ---------------------------------------------------------------------
def _now_iso() -> str:
    """带本地时区偏移的 ISO8601 时间戳。"""
    return datetime.now(timezone.utc).astimezone().isoformat(timespec="seconds")
def load_config(path: Path) -> dict[str, Any]:
    if not path.exists():
        print(f"[错误] 配置文件不存在：{path}", file=sys.stderr)
        sys.exit(1)
    with path.open("r", encoding="utf-8") as fh:
        return yaml.safe_load(fh) or {}
def parse_args(argv: Iterable[str] | None = None) -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="探海灵眸 Oceanus — 边缘感知模拟器",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=(
            "示例：\n"
            "  python simulator.py --scenario demo --event-interval 3 --loop\n"
            "      # 演示：8fps 抽帧 + 3 秒冷却，最快看到事件\n"
            "  python simulator.py --scenario stress --event-interval 1\n"
```

```
            "      # 压测：放宽确认门槛 + 1 秒冷却，压派单引擎\n"
            "  python simulator.py --scenario regression --seed 42 --duration 60\n"
            "      # 回归：固定种子，结果可复现\n"
            "  python simulator.py --dry-run --duration 10\n"
            "      # 只看报文，不连 Broker\n"
            "  python simulator.py --devices CAM-MABI-01 --dup-rate 0.3\n"
            "      # 验证平台幂等判重\n"
        ),
    )
    parser.add_argument("--config", default=str(DEFAULT_CONFIG), help="配置文件路径")
    parser.add_argument(
        "--scenario", default="demo", choices=["demo", "stress", "regression"], help="运行场景"
    )
    parser.add_argument("--host", default=os.getenv("MQTT_HOST", "localhost"), help="MQTT Broker 地…
    parser.add_argument("--port", type=int, default=int(os.getenv("MQTT_PORT", "1883")), help="MQTT…
    parser.add_argument(
        "--interval",
        type=float,
        default=0.125,
        help="抽帧间隔秒数（模拟摄像头帧率），默认 0.125 = 8fps",
    )
    parser.add_argument(
        "--event-interval",
        type=float,
        default=None,
        help="同一目标的最小上报间隔秒数（覆盖配置里的 90s，用于加快演示节奏）",
    )
    parser.add_argument("--loop", action="store_true", help="循环运行不退出")
    parser.add_argument("--duration", type=int, default=0, help="运行时长（秒），0=不限")
    parser.add_argument("--seed", type=int, default=None, help="随机种子（可复现）")
    parser.add_argument("--devices", default=None, help="只模拟指定设备，逗号分隔")
    parser.add_argument("--dup-rate", type=float, default=0.0, help="重复投递比例 0~1")
    parser.add_argument("--net-drop-every", type=int, default=0, help="每 N 轮模拟一次断网")
    parser.add_argument("--dry-run", action="store_true", help="只打印不发送")
    return parser.parse_args(argv)
def main(argv: Iterable[str] | None = None) -> int:
    args = parse_args(argv)
    _require_deps()          # 只在真正运行时检查依赖，便于纯逻辑单测
    config = load_config(Path(args.config))
    config = apply_scenario(args, config)
    # 命令行显式传入的 Broker 地址覆盖配置文件
    if args.host != "localhost":
        config.setdefault("mqtt", {})["host"] = args.host
    if args.port != 1883:
        config.setdefault("mqtt", {})["port"] = args.port
    # --loop 语义：不设 --duration 时就是无限循环（run() 内部靠 _stop 退出）；
    #          设了 --duration 则跑完一轮后重新开始，直到 Ctrl+C。
    if args.loop:
        args.duration = 0
    sim = EdgeSimulator(args, config)
```

```
    def _sigterm(_signum, _frame):
        sim._stop = True
    try:
        signal.signal(signal.SIGTERM, _sigterm)
    except (ValueError, OSError):
        pass   # 非主线程环境（如测试）无法注册信号
    sim.run()
    return 0
if __name__ == "__main__":
    sys.exit(main())
# ===== edge/detector/detector.py =====
"""OpenCV 海漂垃圾检测器：不依赖训练数据的识别方案。
为什么有这个模块
----------------
原本的识别路线是 YOLO11s + ONNX，但它的前置依赖是数据集：
`ml/configs/seasight.yaml` 的 stats 至今是 0（一张图都没采）。
采集 → 标注 → 训练 → 导出 → 边缘部署，整条链 3~4 周且受天气与出海排期
支配，任何一环卡住全线停摆。
而 OpenCV 传统视觉方案的数据依赖是 0：不需要一张标注图，
今天写完明天就能在真实画面上跑出真实检测框。
它靠什么把"不准"这件事兜住
---------------------------
下游的时序校验器（`edge/simulator/simulator.py` 的 `TemporalValidator`）。
设计文档 §2.3 写得很清楚：原始检测 5772 条 → 确认事件 52 条，抑制率 42%。
也就是说，架构上本来就是按"检测器会误报"设计的：本模块只是把
一个高质量检测器换成一个低质量但零成本的检测器，误报由同一道闸压住。
真要换回 YOLO 怎么办
--------------------
本模块的输出契约与 `backend/app/services/ai/server.py` 的 detections
完全一致（`{class, confidence, bbox:[x1,y1,x2,y2]}`），
下游（时序校验 → MQTT 上报 → 派单 → 大屏）一行都不用改。
换模型只要换掉这一个类的 `detect()`。
输出契约
--------
    [{"class": "foam", "confidence": 0.72, "bbox": [x1, y1, x2, y2]}, ...]
`bbox` 是设备原始分辨率下的像素坐标、原点左上，与 mqtt-topics.md 一致。
"""
from __future__ import annotations
import math
from typing import Any
import cv2
import numpy as np
# 类别定义：顺序即类别索引，必须与以下三处逐项一致：
#   1. ml/configs/seasight.yaml           的 names
#   2. backend/app/services/ai/server.py  的 CLASS_NAMES
#   3. docs/software-design.md            的类别定义
# 第四处定义很容易与前四处漂移，所以 tests 里有断言把这几处钉在一起。
CLASS_NAMES = ["foam", "plastic", "fishing_gear", "other"]
# 参数默认值：必须与 edge/config.yaml 的 detector 段逐项一致。
# 项目踩过的坑（见 docs/decisions.md）：默认值散落在代码里，
```

```
# 而 config.yaml 里改了却因为实例化时漏传不生效：且不报错。



# 所以这里每一个默认值都在测试里与 config.yaml 对账。
DEFAULT_CONFIG: dict[str, Any] = {
    "background": {
        "history": 300,
        "dist2_threshold": 300.0,
        "detect_shadows": True,
    },
    "color": {
        "enabled": True,
        "v_min": 150,
        "s_max": 70,
        # 暗色通道：废旧渔网/湿缆绳比海面暗得多，而 KNN 背景建模会把
        # "比背景暗"的东西标成阴影(127)，在阈值化时被剔掉——
        # 实测暗色细长条的检测框因此整帧为空。所以必须单独一路抓暗目标。
        "dark_enabled": True,
        "dark_v_max": 70,
    },
    "morph_open": 3,
    "morph_close": 7,
    "min_area": 40,
    "max_area": 40000,
    "min_solidity": 0.35,
    "max_aspect": 8.0,
    "classifier": {
        "fishing_gear": {"aspect_min": 2.5, "solidity_max": 0.80, "v_max": 170},
        "foam": {"s_max": 70, "v_min": 150, "circularity_min": 0.20},
        "plastic": {"s_min": 70},
    },
    "confidence": {
        "base": 0.55,
        "span": 0.40,
        "min": 0.30,
        "max": 0.95,
        "glare": {
            "v_threshold": 245,
            "area_max": 400,
            "circularity_max": 0.55,
            "penalty": 0.25,
        },
        "size": {
            "small_factor": 0.4,
            "small_times": 2,
            "large_factor": 0.5,
            "large_ratio": 0.5,
        },
    },
}
def _get(config: dict[str, Any] | None, section: str, key: str) -> Any:
    """从配置取值，缺失或配置未给该段时回落到默认。
```

```
    为什么逐键回退而不是整体 `config or DEFAULT_CONFIG`：


    真实边缘盒上的 config.yaml 很可能是旧版（少几个新加的键），
    整体回退会让运维在 yaml 里改的参数静默失效——
    这正是本项目反复出现的缺陷形状。
    """
    if config:
        sec = config.get(section)
        if isinstance(sec, dict) and key in sec:
            return sec[key]
    return DEFAULT_CONFIG[section][key]
def _top(config: dict[str, Any] | None, key: str) -> Any:
    """取 detector 段下的顶层键（不在子段里的那些）。"""
    if config and key in config:
        return config[key]
    return DEFAULT_CONFIG[key]
def _clip01(value: float) -> float:
    return 0.0 if value < 0.0 else (1.0 if value > 1.0 else float(value))
def _clamp(value: float, low: float, high: float) -> float:
    return low if value < low else (high if value > high else value)
class CvDetector:
    """OpenCV 海漂垃圾检测器。
    用法（真实边缘盒）：
        det = CvDetector(config["detector"])
        for frame in stream:
            for d in det.detect(frame):
                ...  # 送 TemporalValidator
    两路候选来源（取并集）：
      1. 背景减除：抓新出现的、在动的东西
      2. 颜色通道：抓白亮的泡沫，即使它停着不动
    为什么必须两路：只用背景减除时，一片随浪停在镜头前的泡沫会在
    几十帧内被学成"背景"，从此再也不报警；而泡沫恰恰是本项目
    最需要抓的类别（连江养殖区最突出、治理优先级最高）。
    """
    def __init__(
        self,
        config: dict[str, Any] | None = None,
        roi: list[list[float]] | None = None,
    ) -> None:
        self.config = config or {}
        self._roi = roi or []
        self._bg = cv2.createBackgroundSubtractorKNN(
            history=int(_get(config, "background", "history")),
            dist2Threshold=float(_get(config, "background", "dist2_threshold")),
            detectShadows=bool(_get(config, "background", "detect_shadows")),
        )
        self._kernel_open = self._kernel(_top(config, "morph_open"))
        self._kernel_close = self._kernel(_top(config, "morph_close"))
        self._frame_index = 0
        self._roi_mask: np.ndarray | None = None
    @staticmethod
```

```
def _kernel(size: int) -> np.ndarray:
    size = max(1, int(size))
    return np.ones((size, size), np.uint8)
# ------------------------------------------------------------------
# 主入口
# ------------------------------------------------------------------
def detect(self, frame: np.ndarray) -> list[dict[str, Any]]:
    """对一帧做检测，返回契约格式的检测列表。"""
    if frame is None or frame.size == 0:
        return []
    self._frame_index += 1
    h, w = frame.shape[:2]
    mask = self._foreground_mask(frame)
    mask = self._apply_roi(mask, (h, w))
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    results: list[dict[str, Any]] = []
    for contour in contours:
        det = self._build_detection(frame, contour)
        if det is not None:
            results.append(det)
    results.sort(key=lambda d: d["confidence"], reverse=True)
    return results
def reset(self) -> None:
    """重置背景模型（摄像头云台转动、昼夜切换后必须调用）。"""
    self._bg = cv2.createBackgroundSubtractorKNN(
        history=int(_get(self.config, "background", "history")),
        dist2Threshold=float(_get(self.config, "background", "dist2_threshold")),
        detectShadows=bool(_get(self.config, "background", "detect_shadows")),
    )
    self._frame_index = 0
    self._roi_mask = None
# ------------------------------------------------------------------
# 候选区域
# ------------------------------------------------------------------
def _foreground_mask(self, frame: np.ndarray) -> np.ndarray:
    """背景减除 ∪ 白亮颜色通道。"""
    fg = self._bg.apply(frame)
    # KNN 把阴影标成 127（detectShadows=True），只取 255 才是真前景。
    #   不剔阴影的话，船影、云影会被当成一大片目标，误报直接翻倍。
    _, fg_bin = cv2.threshold(fg, 200, 255, cv2.THRESH_BINARY)
    if not bool(_get(self.config, "color", "enabled")):
        combined = fg_bin
    else:
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
        # 白亮通道：泡沫是白的、亮的，且可能停着不动
        white = cv2.inRange(
            hsv,
            np.array([0, 0, int(_get(self.config, "color", "v_min"))], np.uint8),
            np.array(
                [179, int(_get(self.config, "color", "s_max")), 255], np.uint8
```

```
            ),
        )
        combined = cv2.bitwise_or(fg_bin, white)
        # 暗色通道：渔网/绳索比海面暗，且会被背景建模当成阴影剔掉
        if bool(_get(self.config, "color", "dark_enabled")):
            dark = cv2.inRange(
                hsv,
                np.array([0, 0, 0], np.uint8),
                np.array(
                    [179, 255, int(_get(self.config, "color", "dark_v_max"))],
                    np.uint8,
                ),
            )
            combined = cv2.bitwise_or(combined, dark)
    opened = cv2.morphologyEx(combined, cv2.MORPH_OPEN, self._kernel_open)
    closed = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, self._kernel_close)
    return closed
def _apply_roi(self, mask: np.ndarray, shape: tuple[int, int]) -> np.ndarray:
    """把画面外的区域（天空、岸边建筑、固定缆绳）裁掉。
    ROI 为空表示全画面。多边形顶点是相对坐标 0~1，
    这样同一份配置在 720p 和 1080p 上都成立。
    """
    if not self._roi:
        return mask
    if self._roi_mask is None or self._roi_mask.shape[:2] != shape:
        h, w = shape
        pts = np.array(
            [[float(x) * w, float(y) * h] for x, y in self._roi], np.int32
        )
        canvas = np.zeros((h, w), np.uint8)
        cv2.fillPoly(canvas, [pts], 255)
        self._roi_mask = canvas
    return cv2.bitwise_and(mask, self._roi_mask)
# ------------------------------------------------------------------
# 单个候选 → 检测
# ------------------------------------------------------------------
def _build_detection(
    self, frame: np.ndarray, contour: np.ndarray
) -> dict[str, Any] | None:
    area = float(cv2.contourArea(contour))
    if area < float(_top(self.config, "min_area")):
        return None
    if area > float(_top(self.config, "max_area")):
        return None
    x, y, w, h = cv2.boundingRect(contour)
    if w <= 0 or h <= 0:
        return None
    perimeter = float(cv2.arcLength(contour, True))
    hull = cv2.convexHull(contour)
    hull_area = float(cv2.contourArea(hull))
```

```
        # 形状特征

        solidity = area / hull_area if hull_area > 0 else 0.0
        circularity = (4.0 * math.pi * area / (perimeter**2)) if perimeter > 0 else 0.0
        aspect = max(w, h) / max(1, min(w, h))
        if solidity < float(_top(self.config, "min_solidity")):
            return None
        if aspect > float(_top(self.config, "max_aspect")):
            return None
        # 颜色/纹理特征（只在轮廓内部取，避免混入背景海面）
        s_mean, v_mean, gray_std = self._color_features(frame, contour)
        cls, match = self._classify(s_mean, v_mean, aspect, solidity, circularity)
        confidence = self._confidence(
            cls, match, area, solidity, circularity, s_mean, v_mean
        )
        img_h, img_w = frame.shape[:2]
        return {
            "class": cls,
            "confidence": round(confidence, 4),
            "bbox": [
                _clamp_int(x, 0, img_w),
                _clamp_int(y, 0, img_h),
                _clamp_int(x + w, 0, img_w),
                _clamp_int(y + h, 0, img_h),
            ],
        }
    @staticmethod
    def _color_features(
        frame: np.ndarray, contour: np.ndarray
    ) -> tuple[float, float, float]:
        """取轮廓内部的平均饱和度、平均明度、灰度标准差。"""
        x, y, w, h = cv2.boundingRect(contour)
        patch = frame[y : y + h, x : x + w]
        if patch.size == 0:
            return 0.0, 0.0, 0.0
        obj_mask = np.zeros((h, w), np.uint8)
        cv2.drawContours(obj_mask, [contour], -1, 255, -1, offset=(-x, -y))
        hsv = cv2.cvtColor(patch, cv2.COLOR_BGR2HSV)
        gray = cv2.cvtColor(patch, cv2.COLOR_BGR2GRAY)
        s_mean = float(cv2.mean(hsv[:, :, 1], mask=obj_mask)[0])
        v_mean = float(cv2.mean(hsv[:, :, 2], mask=obj_mask)[0])
        _, std = cv2.meanStdDev(gray, mask=obj_mask)
        return s_mean, v_mean, float(std[0][0])
    # ------------------------------------------------------------------
    # 分类与打分
    # ------------------------------------------------------------------
    def _classify(
        self,
        s_mean: float,
        v_mean: float,
        aspect: float,
```

```
        solidity: float,
        circularity: float,
    ) -> tuple[str, float]:
        """四类别规则分类，返回 (类别，匹配度 0~1)。
        顺序即优先级，先命中先得：这个顺序是按可分性排的：
        细长暗色（渔网绳索）最独特，先判；白亮团状（泡沫）次之；
        有彩度（塑料）再次；都不像就归 other。
        匹配度不是概率，是"离该类规则中心有多近"的可解释打分，
        现场答辩可以直接把阈值调给评委看。
        """
        fg_cfg = _get(self.config, "classifier", "fishing_gear")
        foam_cfg = _get(self.config, "classifier", "foam")
        plastic_cfg = _get(self.config, "classifier", "plastic")
        # 1) 废旧渔具：暗色 + (细长 或 破碎)
        #    用"或"而不是"且"：湿缆绳是规整的细长条（solidity≈1），
        #      破渔网是破碎的团（长宽比接近 1）。两者都是渔具，
        #      但在形状特征上各占一头，用"且"会漏掉其中一类。
        if v_mean <= float(fg_cfg["v_max"]) and (
            aspect >= float(fg_cfg["aspect_min"])
            or solidity <= float(fg_cfg["solidity_max"])
        ):
            elongation = _clip01((aspect - float(fg_cfg["aspect_min"])) / 3.0)
            broken = _clip01(
                (float(fg_cfg["solidity_max"]) - solidity)
                / float(fg_cfg["solidity_max"])
            )
            return "fishing_gear", _clip01(0.6 * max(elongation, broken) + 0.4 * broken)
        # 2) 泡沫：几乎不带颜色 + 白亮 + 团状
        if (
            s_mean <= float(foam_cfg["s_max"])
            and v_mean >= float(foam_cfg["v_min"])
            and circularity >= float(foam_cfg["circularity_min"])
        ):
            whiteness = _clip01(
                (v_mean - float(foam_cfg["v_min"])) / 60.0
            ) * _clip01((float(foam_cfg["s_max"]) - s_mean) / float(foam_cfg["s_max"]))
            # 过曝惩罚：真实泡沫是"白但没到纯白"（V 约 220~245），
            #   而正午海面高光是纯白过曝（V≈255）。两者在 HSV 里
            #   其他维度几乎一样，只有最顶端这 10 个灰阶能分开。
            over_exposure = _clip01((v_mean - 250.0) / 5.0)
            whiteness *= 1.0 - 0.5 * over_exposure
            roundness = _clip01(circularity / 0.7)
            return "foam", _clip01(0.7 * whiteness + 0.3 * roundness)
        # 3) 塑料：有彩度
        if s_mean >= float(plastic_cfg["s_min"]):
            colorful = _clip01((s_mean - float(plastic_cfg["s_min"])) / 80.0)
            return "plastic", colorful
        return "other", 0.3
    def _confidence(
        self,
```

```
        cls: str,
        match: float,
        area: float,
        solidity: float,
        circularity: float,
        s_mean: float,
        v_mean: float,
    ) -> float:
        """伪置信度 = 基础分 + 匹配度浮动 − 反光惩罚，再夹到 [min, max]。
        为什么需要"伪置信度"：下游 `TemporalValidator` 用
        `min_confidence=0.45` 做第一道过滤。如果所有检测都给 0.9，
        这道过滤就形同虚设（历史上项目里"恒为真"的判断栽过两次）。
        但它压不死反光，这是诚实的边界：
        海面高光与白色泡沫在颜色、形状上高度重合，单帧可分性本来就低。
        真正的区分靠时序：高光随波浪闪现，凑不满 min_hits 就被饿死；
        泡沫持续存在，连续命中。所以反光惩罚的作用是把它的置信度
        显著压到真实泡沫之下（排序靠后），而不是假装能一帧判死。
        """
        conf_cfg = self.config.get("confidence") or DEFAULT_CONFIG["confidence"]
        base = float(conf_cfg.get("base", DEFAULT_CONFIG["confidence"]["base"]))
        span = float(conf_cfg.get("span", DEFAULT_CONFIG["confidence"]["span"]))
        low = float(conf_cfg.get("min", DEFAULT_CONFIG["confidence"]["min"]))
        high = float(conf_cfg.get("max", DEFAULT_CONFIG["confidence"]["max"]))
        shape_quality = _clip01(0.5 * (solidity / 0.9) + 0.5 * (circularity / 0.7))
        size_quality = self._size_quality(area, conf_cfg)
        score = 0.5 * match + 0.3 * shape_quality + 0.2 * size_quality
        value = base + span * score
        # 反光惩罚：极亮 + 很小 + 形状不规整 = 海面高光，不是泡沫
        glare = conf_cfg.get("glare") or DEFAULT_CONFIG["confidence"]["glare"]
        if (
            v_mean >= float(glare["v_threshold"])
            and area <= float(glare["area_max"])
            and circularity <= float(glare["circularity_max"])
        ):
            value -= float(glare["penalty"])
        # 兜底类别整体降一档，避免"什么都不像"的东西靠形状分混过高门限
        if cls == "other":
            value -= 0.1
        return _clamp(value, low, high)
    def _size_quality(self, area: float, conf_cfg: dict[str, Any]) -> float:
        size_cfg = conf_cfg.get("size") or DEFAULT_CONFIG["confidence"]["size"]
        min_area = float(_top(self.config, "min_area"))
        max_area = float(_top(self.config, "max_area"))
        if area < min_area * float(size_cfg["small_times"]):
            return float(size_cfg["small_factor"])
        if area > max_area * float(size_cfg["large_ratio"]):
            return float(size_cfg["large_factor"])
        return 1.0
def _clamp_int(value: float, low: int, high: int) -> int:
    v = int(round(value))
```

```
    return low if v < low else (high if v > high else v)



# ===== edge/main.py =====
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""真实边缘感知程序：取流 → 检测 → 时序校验 → MQTT 上报。
与 `simulator/` 的区别
---------------------
`simulator/simulator.py` 自己造假检测框（用于没有摄像头时驱动平台联调）；
本程序跑真实检测：从 RTSP / 视频文件 / 摄像头取帧，交给检测器
做检测，再走同一套时序校验与上报节流。
检测通道（`detector.backend`，默认 `cv`）
----------------------------------------
- `cv`：`detector.CvDetector`，OpenCV 传统视觉，零数据依赖，主通道；
- `world`：`detector.world_detector.WorldDetector`，开放词汇零样本通道，
  用于演示与对照（现场可加词），默认不进主告警链路。
两个通道的输出契约完全一致，所以时序校验、上报节流、MQTT 报文的代码一行都不用改；
但它们的置信度尺度不同，时序门限必须各用各的（见 `_effective_min_confidence`），
否则 world 通道的检测会在时序环节被静默丢掉。
两者产出的 MQTT 报文结构完全一致，平台侧无法区分。
用法
----
    # 1) 无摄像头自检：用内置合成海面跑 200 帧，只打印不联网
    python main.py --source synthetic --dry-run --max-frames 200
    # 2) 视频文件（录好的码头片段）
    python main.py --source file --file samples/shore.mp4 --dry-run
    # 3) 真实 RTSP 摄像头，上报到平台
    python main.py --source rtsp
    # 4) 调试时把检测框画出来看
    python main.py --source synthetic --show
    # 5) 开放词汇通道 + 现场加词（演示用）
    python main.py --source synthetic --backend world --show \
        --world-prompt "foam buoy" --world-prompt "plastic bag"
"""
from __future__ import annotations
import argparse
import json
import random
import sys
import time
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Iterator
HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(HERE / "detector"))
sys.path.insert(0, str(HERE / "simulator"))
try:
    import cv2
    import numpy as np
except ImportError:  # pragma: no cover
```

```
    print("[错误] 缺少依赖：pip install opencv-python-headless numpy")


    raise SystemExit(1)
from detector import CvDetector  # noqa: E402
from simulator import OutboxBuffer, TemporalValidator  # noqa: E402
DEFAULT_CONFIG = HERE / "config.yaml"
STATE_FILE = HERE / ".edge_state.json"
TOPIC_PREFIX = "marine"
# ---------------------------------------------------------------------
# 工具
# ---------------------------------------------------------------------
def _now_iso() -> str:
    return datetime.now(timezone.utc).isoformat()
def _load_config(path: Path) -> dict[str, Any]:
    try:
        import yaml  # noqa: PLC0415
    except ImportError:
        print("[错误] 缺少 PyYAML：pip install pyyaml")
        raise SystemExit(1)
    return yaml.safe_load(path.read_text(encoding="utf-8")) or {}
def _load_state() -> dict[str, Any]:
    if STATE_FILE.exists():
        try:
            return json.loads(STATE_FILE.read_text(encoding="utf-8"))
        except Exception:  # noqa: BLE001
            pass
    return {}
def _save_state(state: dict[str, Any]) -> None:
    try:
        STATE_FILE.write_text(
            json.dumps(state, ensure_ascii=False, indent=2), encoding="utf-8"
        )
    except Exception as exc:  # noqa: BLE001
        print(f"[警告] 状态写入失败：{exc}")
# ---------------------------------------------------------------------
# 取流
# ---------------------------------------------------------------------
class SyntheticSea:
    """合成海面帧序列：没有摄像头时也能端到端自检。
    为什么要有它：真实视频依赖出海采集，而"这条链路通不通"
    （检测 → 时序 → 报文 → 上报）不应该等到那时才验证。
    它同时也是现场演示的兜底：摄像头坏了也能把流程演完。
    """
    def __init__(self, seed: int = 42, size: tuple[int, int] = (360, 640)) -> None:
        self.rng = np.random.default_rng(seed)
        self.size = size
        self.index = 0
        # 三块"漂浮泡沫"：缓慢横向漂移 + 上下浮动（0.1~0.5 m/s 的量级）
        self.blobs = [
            {"x": 120.0, "y": 150.0, "r": 16, "vx": 1.2, "vy": 0.2},
            {"x": 400.0, "y": 240.0, "r": 13, "vx": -0.9, "vy": 0.15},
```

```
            {"x": 560.0, "y": 110.0, "r": 18, "vx": 0.6, "vy": -0.1},
        ]
    def __iter__(self) -> Iterator[np.ndarray]:
        return self
    def __next__(self) -> np.ndarray:
        h, w = self.size
        frame = np.zeros((h, w, 3), np.uint8)
        frame[:] = (110, 100, 82)
        noise = self.rng.normal(0, 4, frame.shape)
        frame = np.clip(frame.astype(np.int16) + noise, 0, 255).astype(np.uint8)
        for b in self.blobs:
            b["x"] += b["vx"]
            b["y"] += b["vy"]
            if b["x"] > w - 30 or b["x"] < 30:
                b["vx"] *= -1
            cv2.circle(
                frame,
                (int(b["x"]), int(b["y"])),
                b["r"],
                (232, 235, 238),
                -1,
            )
        # 每 40 帧来一次"水花"（瞬态噪声，不该被确认成事件）
        if self.index % 40 == 7:
            x = int(self.rng.integers(60, w - 60))
            y = int(self.rng.integers(60, h - 60))
            cv2.circle(frame, (x, y), 10, (250, 250, 250), -1)
        self.index += 1
        return frame
def open_source(cfg: dict[str, Any], args: argparse.Namespace):
    """按配置/参数打开视频源。"""
    src = args.source or cfg.get("source", {}).get("type", "file")
    if src == "synthetic":
        return SyntheticSea(seed=args.seed)
    if src == "camera":
        return cv2.VideoCapture(int(cfg.get("source", {}).get("camera", 0)))
    if src == "rtsp":
        return cv2.VideoCapture(args.rtsp or cfg["source"]["rtsp"])
    path = args.file or cfg.get("source", {}).get("file")
    if not path:
        print("[错误] 未指定视频文件路径（--file 或 source.file）")
        raise SystemExit(1)
    return cv2.VideoCapture(str(HERE / path if not Path(path).is_absolute() else path))
def next_frame(source) -> np.ndarray | None:
    """统一取帧接口（生成器 / VideoCapture 都支持）。"""
    if isinstance(source, SyntheticSea):
        return next(source, None)
    ok, frame = source.read()
    return frame if ok else None
# ---------------------------------------------------------------------
```

```
# 边缘程序主体


# ---------------------------------------------------------------------
class EdgeRuntime:
    def __init__(self, cfg: dict[str, Any], args: argparse.Namespace) -> None:
        self.cfg = cfg
        self.args = args
        self.detector = self._build_detector(cfg, args)
        self.backend_name = str(getattr(self.detector, "backend_name", "cv"))
        t = cfg.get("temporal", {})
        # 时序门限按通道取，两个通道各用各的（原因见 _effective_min_confidence）
        self.min_confidence = self._effective_min_confidence(cfg)
        self.validator = TemporalValidator(
            window_frames=int(t.get("window_frames", 15)),
            min_hits=int(t.get("min_hits", 3)),
            grid_size=int(t.get("grid_size", 64)),
            min_confidence=self.min_confidence,
            match_distance=float(t.get("match_distance", 60.0)),
            max_misses=int(t.get("max_misses", 5)),
            min_iou=float(t.get("min_iou", 0.25)),
            decimation=int(t.get("decimation", 4)),
        ) if t.get("enabled", True) else None
        rep = cfg.get("reporting", {})
        # 演示时用 --cooldown 3 把 90 秒冷却压到 3 秒，几秒就能看到连续告警；
        # 真实边缘盒保持 90 秒（同一片垃圾不该被反复派单）。
        self.cooldown = float(
            args.cooldown if args.cooldown is not None
            else rep.get("cooldown_seconds", 90)
        )
        self.min_interval = float(
            args.min_interval if args.min_interval is not None
            else rep.get("min_interval_seconds", 2)
        )
        self.max_detections = int(rep.get("max_detections", 20))
        buf = cfg.get("buffer", {})
        self.buffer = OutboxBuffer(max_size=int(buf.get("max_size", 1000)))
        self.state = _load_state()
        self.seq = int(self.state.get("seq", 0))
        self._last_report: dict[tuple, float] = {}
        self._last_any = 0.0
        self.counters = {"frames": 0, "raw": 0, "confirmed": 0, "published": 0,
                         "buffered": 0, "throttled": 0}
        self._client = None
    # MQTT
    def connect(self) -> None:
        if self.args.dry_run:
            print("[DRY-RUN] 不连接 Broker，报文只打印")
            return
        import paho.mqtt.client as mqtt  # noqa: PLC0415
        m = self.cfg.get("mqtt", {})
        dev = self.cfg.get("device", {})
```

```
        client = mqtt.Client(
            mqtt.CallbackAPIVersion.VERSION2,
            client_id=f"edge-{dev.get('device_id', 'unknown')}",
        )
        client.username_pw_set(m.get("username"), m.get("password"))
        if m.get("use_lwt", True):
            client.will_set(self._topic("status"), json.dumps(
                {"device_id": dev.get("device_id"), "online": False, "ts": _now_iso()}
            ), qos=1, retain=True)
        client.connect(m.get("host", "localhost"), int(m.get("port", 1883)),
                       int(m.get("keepalive", 60)))
        client.loop_start()
        self._client = client
        self._publish_status(True)
    def _topic(self, kind: str) -> str:
        dev = self.cfg.get("device", {})
        return (f"{TOPIC_PREFIX}/{self.cfg.get('site_id', 'lianjiang')}"
                f"/{dev.get('device_id')}/{kind}")
    def _publish_status(self, online: bool) -> None:
        dev = self.cfg.get("device", {})
        self._publish(self._topic("status"), {
            "device_id": dev.get("device_id"),
            "online": online,
            "ts": _now_iso(),
        }, qos=1)
    def _publish(self, topic: str, payload: dict[str, Any], qos: int) -> None:
        body = json.dumps(payload, ensure_ascii=False)
        if self.args.dry_run or self._client is None:
            if self.args.dry_run:
                agg = payload.get("aggregate", {})
                print(f"[DRY-RUN] {topic}  seq={payload['seq']}  "
                      f"class={agg.get('main_class')} "
                      f"conf={agg.get('max_confidence')} bbox={payload['detections'][0]['bbox']}")
                self.counters["published"] += 1
                return
            self.counters["buffered"] += 1
            return
        try:
            result = self._client.publish(topic, body, qos=qos)
            if getattr(result, "rc", 0) != 0:
                self.buffer.put(topic, payload, qos)
                self.counters["buffered"] += 1
            else:
                self.counters["published"] += 1
        except Exception as exc:  # noqa: BLE001
            print(f"[警告] 发布失败: {exc} (已缓存) ")
            self.buffer.put(topic, payload, qos)
            self.counters["buffered"] += 1
    # 主循环
    def build_event(self, det: dict[str, Any]) -> dict[str, Any]:
```

```
        """构造与 simulator 完全一致的事件报文。"""

        self.seq += 1
        dev = self.cfg.get("device", {})
        cls = det["class"]
        return {
            "event_id": f"evt_{dev.get('device_id')}_{self.seq:06d}",
            "device_id": dev.get("device_id"),
            "device_type": dev.get("device_type", "shore_camera"),
            "timestamp": _now_iso(),
            "location": {"lng": dev.get("lng"), "lat": dev.get("lat")},
            "detections": [det],
            "aggregate": {
                "main_class": cls,
                "count": 1,
                "max_confidence": det["confidence"],
            },
            "evidence_url": None,
            "model_version": "cv-opencv-v0.1.0",
            "seq": self.seq,
        }
    def _cooldown_ok(self, det: dict[str, Any]) -> bool:
        now = time.time()
        if now - self._last_any < self.min_interval:
            return False
        cell = (self.validator._cell(det["bbox"]) if self.validator else (0, 0))
        key = (det["class"], cell)
        if now - self._last_report.get(key, 0.0) < self.cooldown:
            return False
        self._last_report[key] = now
        self._last_any = now
        return True
    # ------------------------------------------------------------------
    # 检测通道选择
    # ------------------------------------------------------------------
    @staticmethod
    def _build_detector(cfg: dict[str, Any], args: argparse.Namespace) -> Any:
        """按 `detector.backend` 实例化检测器（命令行 `--backend` 优先）。
        设计约束：只在这里加分支，不碰任何检测器内部的实现。
        `CvDetector` 的代码路径（主通道、线上只读挂载、后端 3 处 import）
        必须保持一次都没被改过：这样本改动对现网是零影响。
        """
        detector_cfg = cfg.get("detector") or {}
        backend = str(
            getattr(args, "backend", None) or detector_cfg.get("backend") or "cv"
        ).strip().lower()
        if backend == "cv":
            return CvDetector(config=detector_cfg, roi=cfg.get("roi") or [])
        if backend != "world":
            raise SystemExit(f"[配置错误] 未知的检测通道 {backend!r}（可选：cv / world）")
        # 延迟导入：cv 通道的代码路径不因 world 通道的存在而多一个导入点
```

```
        from world_detector import WorldDetector  # noqa: PLC0415



        world_cfg = dict(detector_cfg.get("world") or {})
        extra_prompts = [str(p) for p in (getattr(args, "world_prompt", None) or [])]
        if extra_prompts:
            world_cfg["prompt_classes"] = list(
                world_cfg.get("prompt_classes") or []
            ) + extra_prompts
        return WorldDetector(config=world_cfg, roi=cfg.get("roi") or [])
    def _effective_min_confidence(self, cfg: dict[str, Any]) -> float:
        """当前通道应当使用的时序门限。
        这是接入开放词汇通道时最容易埋雷的一处：
        `temporal.min_confidence` 的 0.45 是给 cv 通道的伪置信度
        （范围 0.30~0.95）标定的；开放词汇模型的输出分数低一个量级
        （本仓库历史实测最高 0.184）。若共用 0.45，
        world 通道的检测会在时序环节被全部丢掉，而且不报任何错。
        做法：谁有自己标定值就用谁的（`WorldDetector.temporal_min_confidence`），
        否则回落配置里的 temporal 门限：也就是 cv 通道的现网行为。
        """
        world_gate = getattr(self.detector, "temporal_min_confidence", None)
        if world_gate is not None:
            return float(world_gate)
        return float(cfg.get("temporal", {}).get("min_confidence", 0.45))
    def run(self) -> int:
        # 先确认检测通道真的能跑，再连 Broker。
        #   不可用时必须明确失败：本通道不可用的现象与"画面里没有垃圾"
        #   完全一样（都是零检测），静默继续会让人以为系统在正常工作。
        #
        #   两步都要问，缺一不可：
        #   1) available：依赖与权重在不在（便宜的存在性探测，不加载模型）；
        #   2) load()：模型真的建得起来吗。
        #   只问第 1 步会漏掉"依赖齐、权重在，但模型加载失败"这一类
        #    （实测案例：torch<2.6 时 ultralytics 拒载 .pt，此时 available 为 True）。
        if not bool(getattr(self.detector, "available", True)):
            print(f"[启动失败] 检测通道 {self.backend_name} 不可用："
                  f"{getattr(self.detector, 'last_error', '未知原因')}")
            print("           这不是「画面里没有垃圾」，而是这一路根本没跑起来。")
            print("           请补齐依赖/权重，或把 detector.backend 改回 cv。")
            return 3
        loader = getattr(self.detector, "load", None)
        if callable(loader) and not loader():
            print(f"[启动失败] 检测通道 {self.backend_name} 的模型加载失败："
                  f"{getattr(self.detector, 'last_error', '未知原因')}")
            print("           依赖与权重都在，但模型本身没能建起来："
                  "这同样不是「画面里没有垃圾」。")
            print("           请按上面的原因修运行时，或把 detector.backend 改回 cv。")
            return 3
        src_cfg = self.cfg.get("source", {})
        log_every = int(src_cfg.get("log_every", 100))
        source = open_source(self.cfg, self.args)
        self.connect()
```

```
    print(f"[启动] 设备={self.cfg.get('device', {}).get('device_id')} "
          f"源={self.args.source or src_cfg.get('type')} "
          f"检测器={self.backend_name} 时序门限={self.min_confidence}")
    try:
        while True:
            if self.args.max_frames and self.counters["frames"] >= self.args.max_frames:
                break
            frame = next_frame(source)
            if frame is None:
                break
            self.counters["frames"] += 1
            detections = self.detector.detect(frame)
            self.counters["raw"] += len(detections)
            if self.validator is None:
                confirmed = [
                    d for d in detections
                    if d["confidence"] >= self.min_confidence
                ]
            else:
                confirmed = self.validator.push(detections)
                # 抽帧推理下 miss 衰减必须按被跳过的帧推进
                self.validator.sweep(self.validator.decimation)
            for det in confirmed[: self.max_detections]:
                if not self._cooldown_ok(det):
                    self.counters["throttled"] += 1
                    continue
                self.counters["confirmed"] += 1
                self._publish(self._topic("event"), self.build_event(det), qos=1)
            if self.args.show:
                for d in detections:
                    x1, y1, x2, y2 = d["bbox"]
                    cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
                    cv2.putText(frame, f"{d['class']} {d['confidence']:.2f}",
                                (x1, max(12, y1 - 6)), cv2.FONT_HERSHEY_SIMPLEX,
                                0.45, (0, 255, 0), 1)
                cv2.imshow("seasight-edge", frame)
                if cv2.waitKey(1) & 0xFF == ord("q"):
                    break
            if log_every and self.counters["frames"] % log_every == 0:
                print(f"  帧={self.counters['frames']} 原始检测={self.counters['raw']} "
                      f"确认={self.counters['confirmed']} 节流={self.counters['throttled']}")
    except KeyboardInterrupt:
        print("\n[中断] 收到 Ctrl+C")
    finally:
        if isinstance(source, cv2.VideoCapture):
            source.release()
        if self._client is not None:
            self._publish_status(False)
            self._client.loop_stop()
            self._client.disconnect()
```

```
            self.state["seq"] = self.seq




            _save_state(self.state)
            if self.args.show:
                cv2.destroyAllWindows()
        st = self.validator.stats if self.validator else {}
        suppressed = st.get("suppressed", 0)
        fed = st.get("fed", 0)
        print("\n[结束]")
        print(f"  处理帧数  : {self.counters['frames']}")
        print(f"  原始检测  : {self.counters['raw']}")
        print(f"  确认事件  : {self.counters['confirmed']}")
        print(f"  节流丢弃  : {self.counters['throttled']}")
        print(f"  已发布    : {self.counters['published']}")
        if fed:
            print(f"  误报抑制率 : {suppressed / fed:.1%}")
        # 开放词汇通道的可解释计数：把"为什么少了检测框"一次说清。
        # 没有这组数字，'提示词没归宿'与'门限太高'的现象一模一样（某类恒为 0）。
        if hasattr(self.detector, "stats"):
            ds = self.detector.stats()
            print(f"  [{self.backend_name} 通道] 调用={ds['calls']} 原始={ds['raw']} "
                  f"保留={ds['kept']} 低置信丢弃={ds['dropped_low_conf']} "
                  f"ROI 丢弃={ds['dropped_roi']} 非法框={ds['dropped_bad_box']} "
                  f"回落 other={ds['fallback_other']}")
            if ds.get("unmapped_prompts"):
                print(f"  [注意] 这些提示词没有归宿、会回落 other：{ds['unmapped_prompts']}")
        return 0
def main() -> int:
    parser = argparse.ArgumentParser(description="探海灵眸边缘感知程序（真实检测：cv / world 双通道…
    parser.add_argument("--config", default=str(DEFAULT_CONFIG), help="配置文件路径")
    parser.add_argument("--source", choices=["rtsp", "file", "camera", "synthetic"],
                        help="覆盖配置里的视频源类型")
    parser.add_argument("--file", help="视频文件路径（--source file 时用）")
    parser.add_argument("--rtsp", help="RTSP 地址（--source rtsp 时用）")
    parser.add_argument("--max-frames", type=int, help="最多处理多少帧后退出（自检用）")
    parser.add_argument("--seed", type=int, default=42, help="合成海面的随机种子")
    parser.add_argument("--cooldown", type=float,
                        help="覆盖上报冷却秒数（演示时设 3~5，默认读配置 90）")
    parser.add_argument("--min-interval", type=float,
                        help="覆盖任意两次上报的最小间隔秒数（默认读配置 2）")
    parser.add_argument("--dry-run", action="store_true", help="不连 Broker，只打印报文")
    parser.add_argument("--show", action="store_true", help="弹窗显示检测框（调试用）")
    parser.add_argument("--backend", choices=["cv", "world"],
                        help="覆盖检测通道（默认读 detector.backend，现网为 cv）")
    parser.add_argument("--world-prompt", action="append", metavar="PROMPT",
                        help="给开放词汇通道追加提示词（可重复），现场加词演示用")
    args = parser.parse_args()
    cfg = _load_config(Path(args.config))
    return EdgeRuntime(cfg, args).run()
if __name__ == "__main__":
    raise SystemExit(main())
```